BECE407P ASIC DESIGN

Module - 4 & 5: Timing Analysis

Dr. PRITAM BHATTACHARJEE

Assistant Professor (Senior Grade 2)

School of Electronics Engineering (SENSE)

Vellore Institute of Technology – Chennai

pritam.bhattacharjee@vit.ac.in, +91 8132863424

Contents

Introduction

Timing analysis is the process of verifying that all signals in an integrated circuit (IC) meet required timing constraints, so data is reliably transferred and processed without errors.

- **Validates Signal Propagation:** Ensures that all signals in the circuit arrive at their destinations within specified timing windows, preventing data corruption and ensuring the intended functionality of the chip.
- **Classifies Timing Analysis Types:**
  - **Static Timing Analysis (STA):** Checks the worst-case delays of all signal paths without using input vectors, making the process fast and comprehensive.
  - **Dynamic Timing Analysis:** Uses the actual input patterns to emulate circuit behavior, providing more realistic but slower verification.
- **Checks Critical Timing Parameters:**
  - **Setup time:** Minimum period data must be stable before the clock edge.
  - **Hold time:** Minimum period data must be stable after the clock edge.
  - **Propagation delay:** Time taken by the signals to move through the gates and interconnects.

Timing Parameter Definitions

- Timing parameters define the time-based constraints in digital circuits, such as propagation delays, setup/hold times, and clock periods.

- These parameters ensure that signals arrive at their destinations at the correct times, preventing timing faults that could lead to system malfunction.

- Key timing parameters: **Clock period**, **setup time**, **hold time**, **propagation delay**, **clock skew**, and **clock jitter**.

Basics of Timing Analysis

The basis of all timing analysis is the “Clock” and “Sequential component” (flipflop).

Following are few things related to clock and flipflop which we usually want to take care during Timing Analysis:

Clock related:

- It must be well understood parametrically and must be glitch-free.
- Any clocks that are generated by the logic are clean, are of bounded period and duty cycle, and of a known phase relationship to other clock signals of interest.
- When “passing” data from one clock edge to the other, ensure that the worst-case duty cycle is used for the calculation.

FlipFlop related:

- Make sure that all parameters of flipflops always met. The only exception is when synchronizers are used to synchronize asynchronous signals.
- All setup and hold times are met for the earliest/latest arrival times for the clock.
- Setup times are generally calculated by designers and suitable margins can be demonstrated under test. Hold times, however, are not frequently calculated by designers.
- When passing data from one clock to another, ensure that there is either known phase relationship which will guarantee meeting setup and hold times or that the circuits are properly synchronized.

Static Timing Analysis

It is a method of validating the timing performance of a design by checking all possible paths for timing violations under worst-case conditions. It considers the worst possible delay through each logic element, but not the logical operation of the circuit.

The Way STA is performed:

- Design is broken down into sets of timing paths,
- Calculates the signal propagation delay along each path,
- And checks for violations of timing constraints inside the design and at the input/output interface.

Timing Paths: They can be divided as per the type of signals (e.g., clock signal, data signal, etc.)

Types of Paths for Timing Analysis:

- Data Path
- Clock Path
- Clock Gating Path
- Asynchronous Path
- Critical Path
- False Path
- Multi-cycle path
- Single Cycle path
- Launch path
- Capture path

Static Timing Analysis

Each timing path has a "**Start Point**" and an "**End Point**" whose definition vary as per the type of the timing path.
E.g.: for the Datapath – The start point is a place in the design where data is launched by a clock edge. The data is propagated through combinational logic in the path and then captured at the endpoint by another clock edge.
Start Point and **End Point** are different for each type of paths. It's very important to understand this clearly to analyze the Timing Analysis report and fix the timing violation.

Data path

Start Point

Input port of the design (because the input data can be launched from some external source).
Clock pin of the flipflop/latch/memory (sequential cell).

End Point

Data input pin of the flipflop/latch/memory (sequential cell).
Output port of the design (because the output data can be captured by some external sink).

Clock Path

Start Point

Clock input port.

End Point

Clock pin of the flipflop/latch/memory (sequential cell).

Static Timing Analysis

Each timing path has a "**Start Point**" and an "**End Point**" whose definition vary as per the type of the timing path.
E.g.: for the Datapath – The start point is a place in the design where data is launched by a clock edge. The data is propagated through combinational logic in the path and then captured at the endpoint by another clock edge.
Start Point and **End Point** are different for each type of paths. It's very important to understand this clearly to analyze the Timing Analysis report and fix the timing violation.

Clock Gating path

Start Point

Input port of the design.

End Point

Input port of the clock-gating element.

Asynchronous Path

Start Point

Input port of the design.

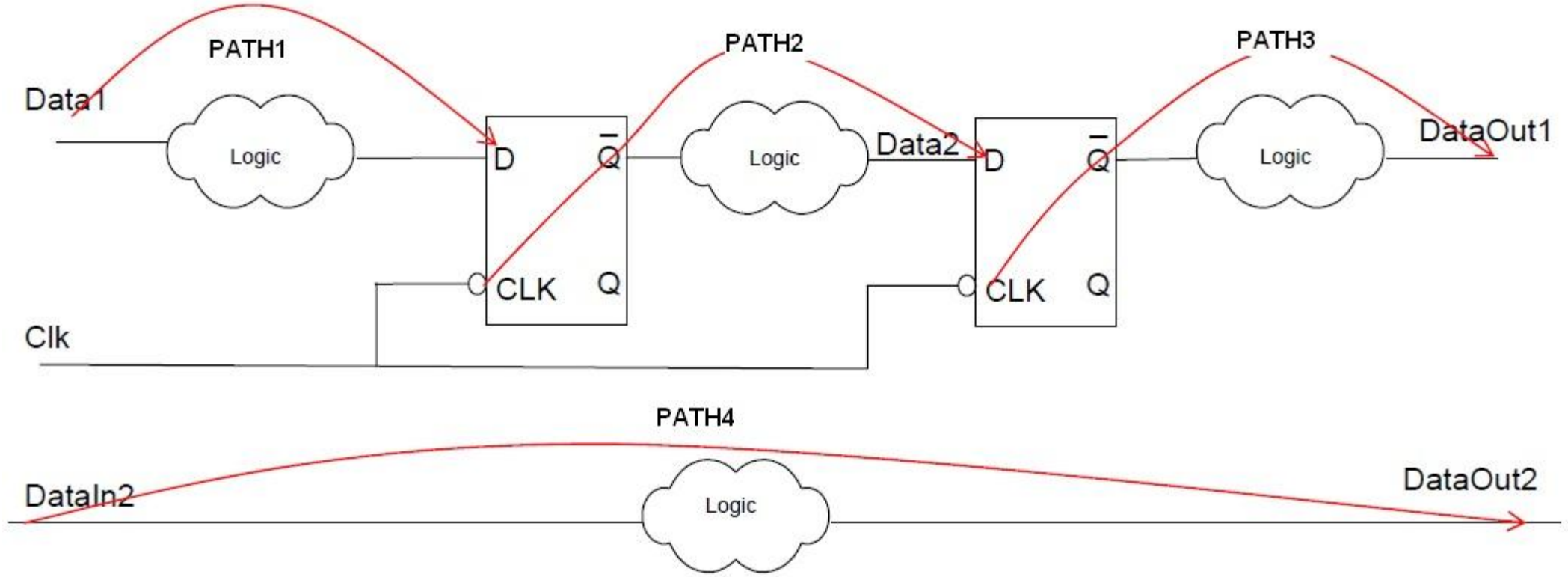
End Point

Set/Reset/Clear of the flipflop/latch/memory (sequential cell).

Static Timing Analysis

Data Paths:

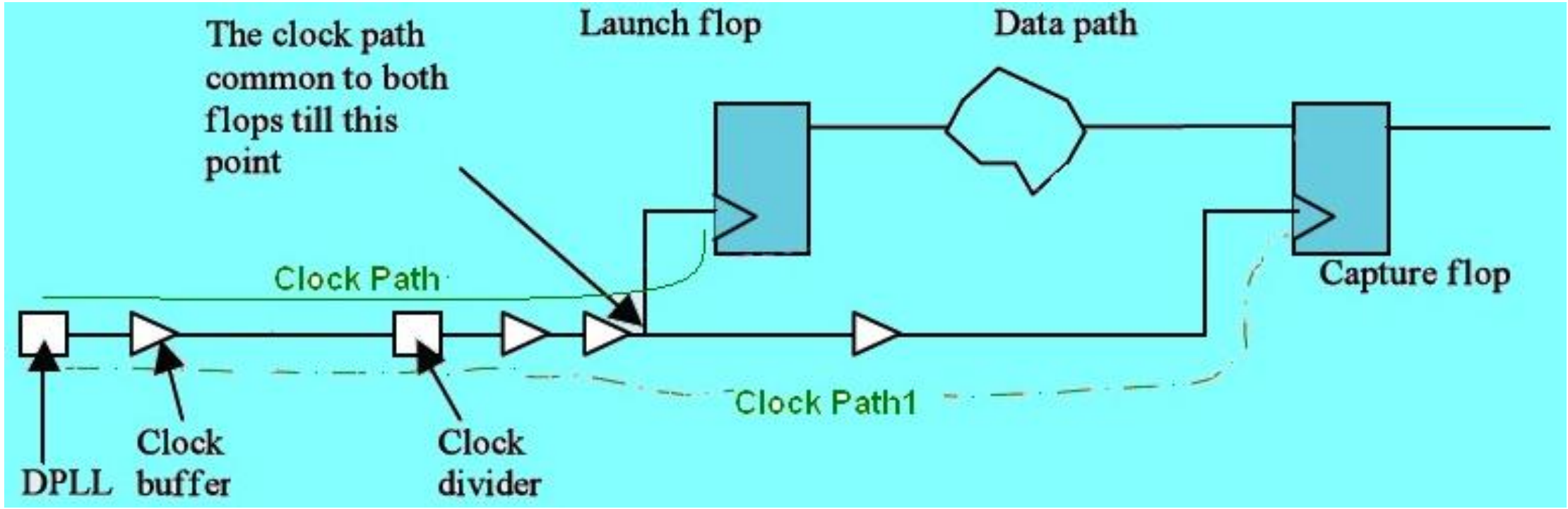
PATH1 – starts at an input port and ends at the data input of a sequential element. (**Input port to Register**)
PATH2 – starts at the clock pin of a sequential element and ends at the data input of the same. (**Register to Register**)
PATH3 – starts at the clock pin of a sequential element and ends at an output port. (**Register to Output port**)
PATH4 – starts at an input port and ends at an output port. (**Input port to Output port**)



Static Timing Analysis

Clock Path:

For Clock path, the **Start Point** is from the input port/pin of the design which is specific for the Clock input and the **End Point** is the clock pin of a sequential element. In between the Start Point and End Point there may be lots of Buffers/Inverters/ClockDividers

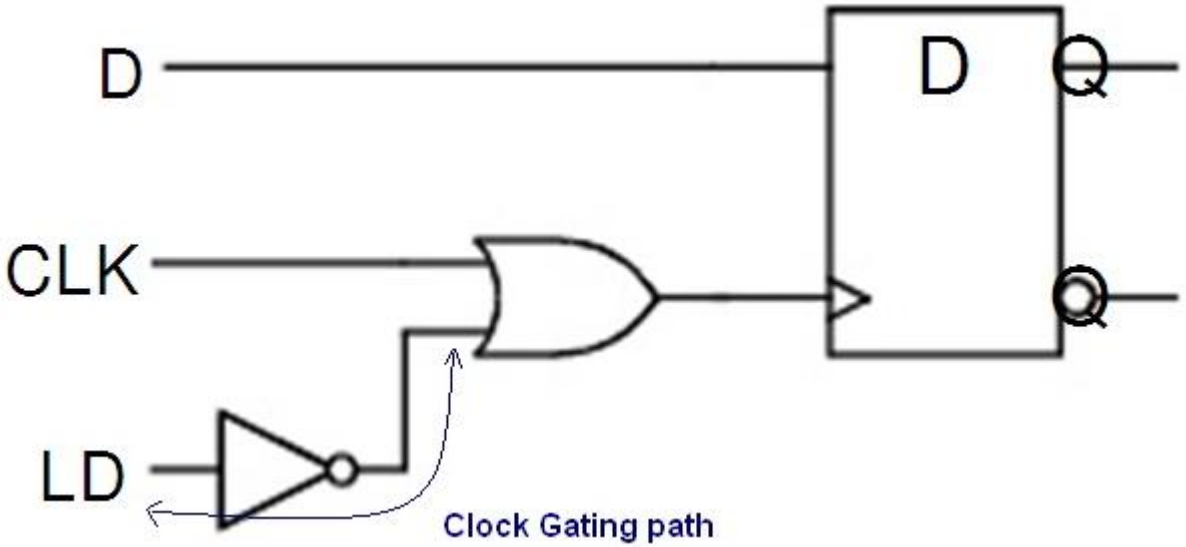


Static Timing Analysis

Clock Gating Path:

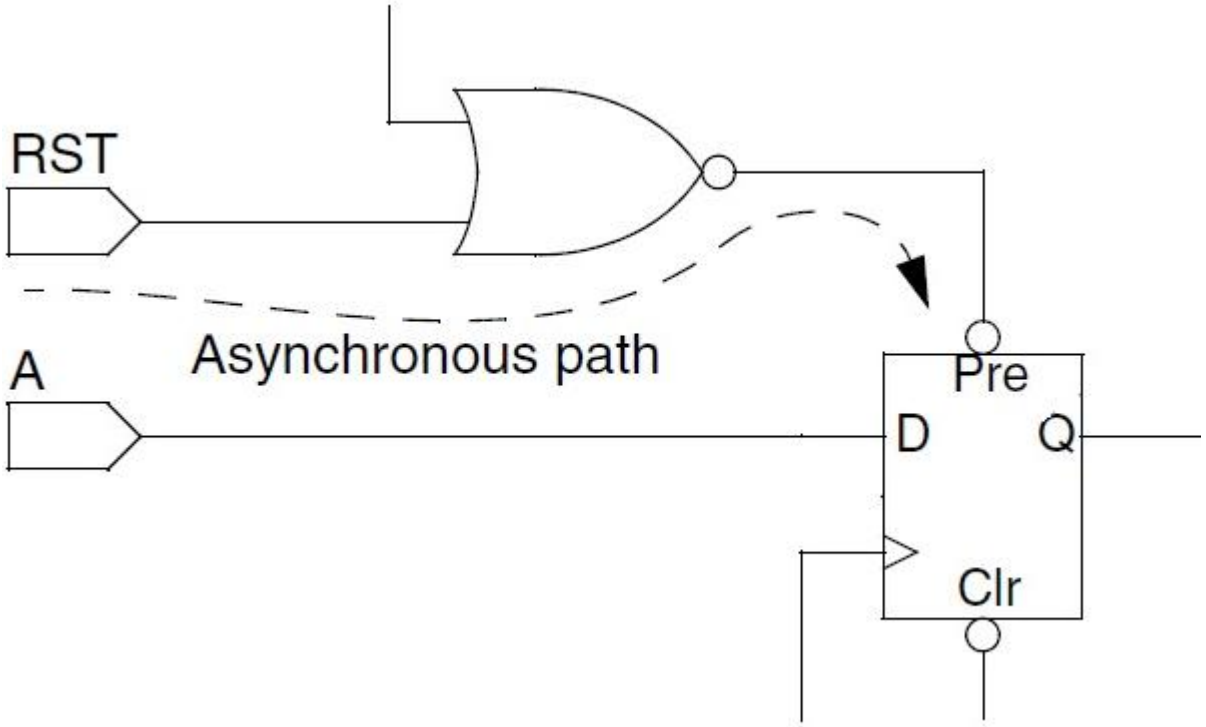
Clock path may be passed through a "**gated element**" to achieve additional advantages. In this case, characteristics and definitions of the clock change accordingly. We call this type of clock path as "**gated clock path**".

"**LD**" pin is not a part of any clock but it is using for gating the original CLK signal. Such type of paths are neither a part of Clock path nor of Data path because as per the Start Point and End Point definition of these paths, its different. So, such type of paths are part of **Clock Gating path**.



Static Timing Analysis

Asynchronous Path:
A path from an input port to an asynchronous set or clear pin of a sequential element.



As you know that the functionality of set/reset pin is independent from the clock edge. Its level triggered pins and can start functioning at any time of data. So, in other way, we can say that this path is not in synchronous with the rest of the circuit and that's the reason we are saying such type of path an **Asynchronous Path**.

Static Timing Analysis

Critical Path:

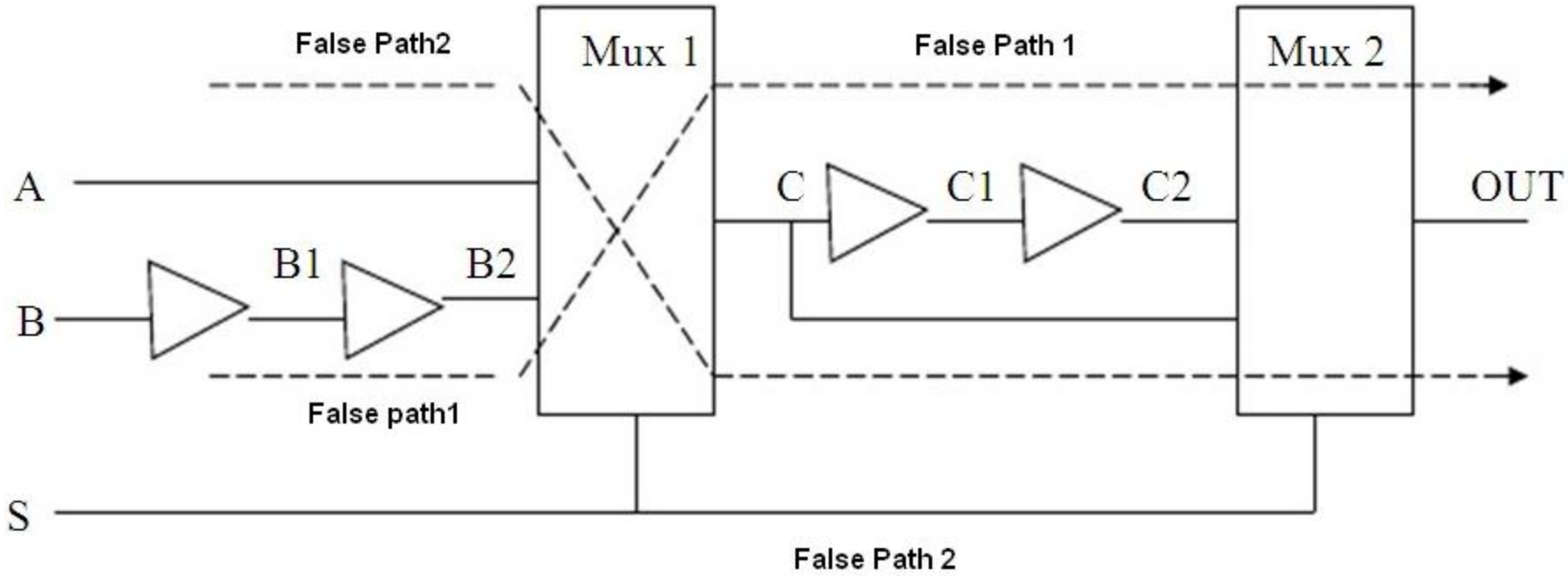
A path which creates **Longest Delay** is the critical path.

- **Critical paths** are **timing-sensitive functional paths** because of the timing of these paths is critical, hence, no additional gates are allowed to be added to the path, to prevent increasing the delay of the critical path.
- Timing critical path are those path that do not meet your timing budget. What normally happens is that after synthesis the tool will give you a number of path which have a negative slack. The first thing you would do is make sure those path are not false or multi-cycle since in that case you can just ignore them.

Let's take an example,

- The STA tool will add the delay contributed from all the logic connecting the 'Q' output of one flipflop to the 'D' input of the next (including the CLK->Q of the first flipflop), and then compare it against the defined clock period of the CLK pins (assuming both flipflops are on the same clock, and taking into account the setup time of the second flipflop and the clock skew).
- This should be strictly less than the clock period defined for that clock.
- If the delay is less than the clock period, then the "*path meets timing*".
- If the delay is greater, then the "*path fails timing*".
- The "**critical path**" is the path out of all the possible paths that either exceeds its constraint by the largest amount, or, if all paths pass, then it is the one that comes closest to failing.

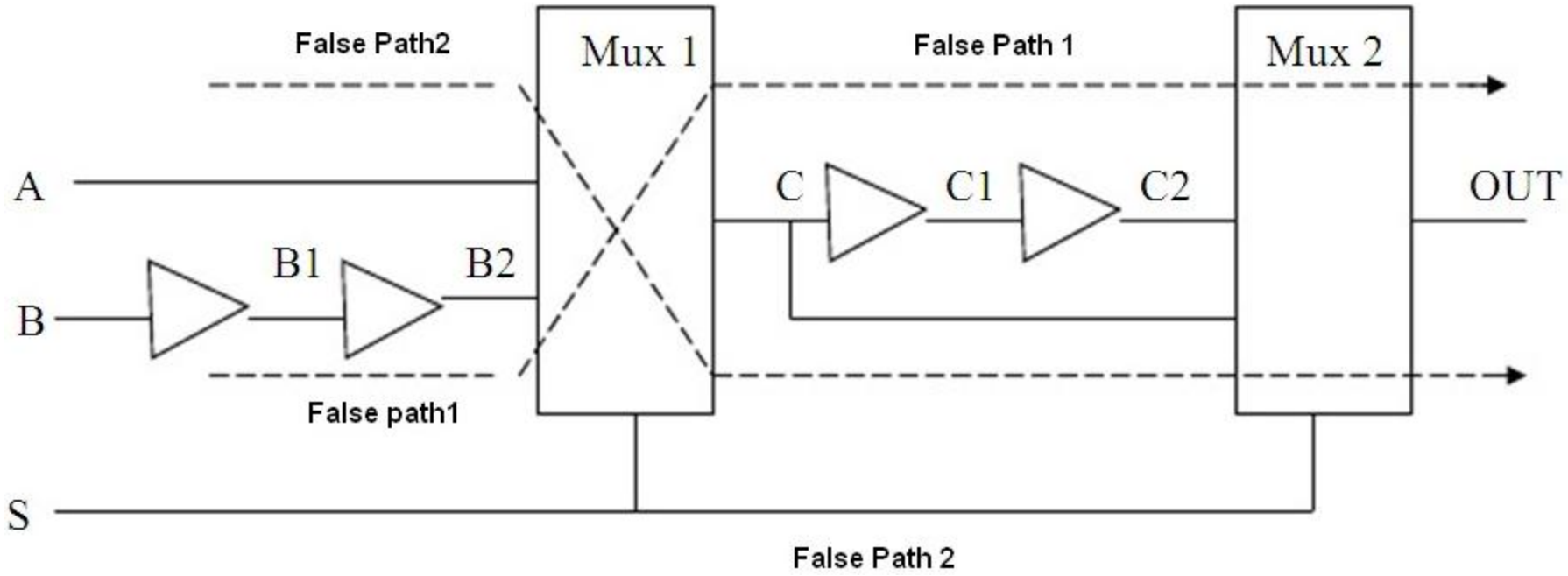
Static Timing Analysis



False Path:

- Physically exist in the design but those are logically/functionally incorrect path. This means that no data is transferred from Start Point to End Point. There may be several reasons of such path present in the design.
- Some time we have to explicitly define/create few false path within the design. For example, setting a relationship between 2 Asynchronous Clocks.
- The goal in STA is to do timing analysis on all "true" timing paths, so "false" paths are excluded from timing analysis.
- Since false path are not exercised during normal circuit operation, they typically don't meet timing specification, considering false path during timing closure can result into timing violations and the procedure to fix would introduce unnecessary complexities in the design.

Static Timing Analysis



False Path:

- There may be few paths in your design which are not critical for timing or masking other paths which are important for timing optimization, or never occur with in normal situation. In such case, to increase the run time and improve the timing result, sometime we have to declare such path as a False path, so that Timing Analysis tool ignore these paths.
- For example, a path between two multiplexed blocks that are never enabled at the same time. The False Path 1 and False path 2 cannot occur at the same time but during optimization it can affect the timing of another path. So, in such scenario, we have to define one of the path as False Path.

Static Timing Analysis

Multi-cycle Path:

- A multi-cycle path is a timing path that is designed to take more than one clock cycle for the data to propagate from the Start Point to the End Point.
- A multi-cycle path is a path that is allowed multiple clock cycles for propagation. Again, it is a path that starts at a timing Start Point and ends at a timing End Point. However, for a multi-cycle path, the normal constraint on this path is overridden to allow for the propagation to take multiple clocks.
- In the simplest example, the Start Point and End Point are flipflops clocked by the same clock. The normal constraint is therefore applied by the definition of the clock; sum of all delays from the CLK arrival at the first flipflop to the arrival at the D of the second clock should take no more than 1 clock period minus the setup time of the second flipflop and adjusted for clock skew.

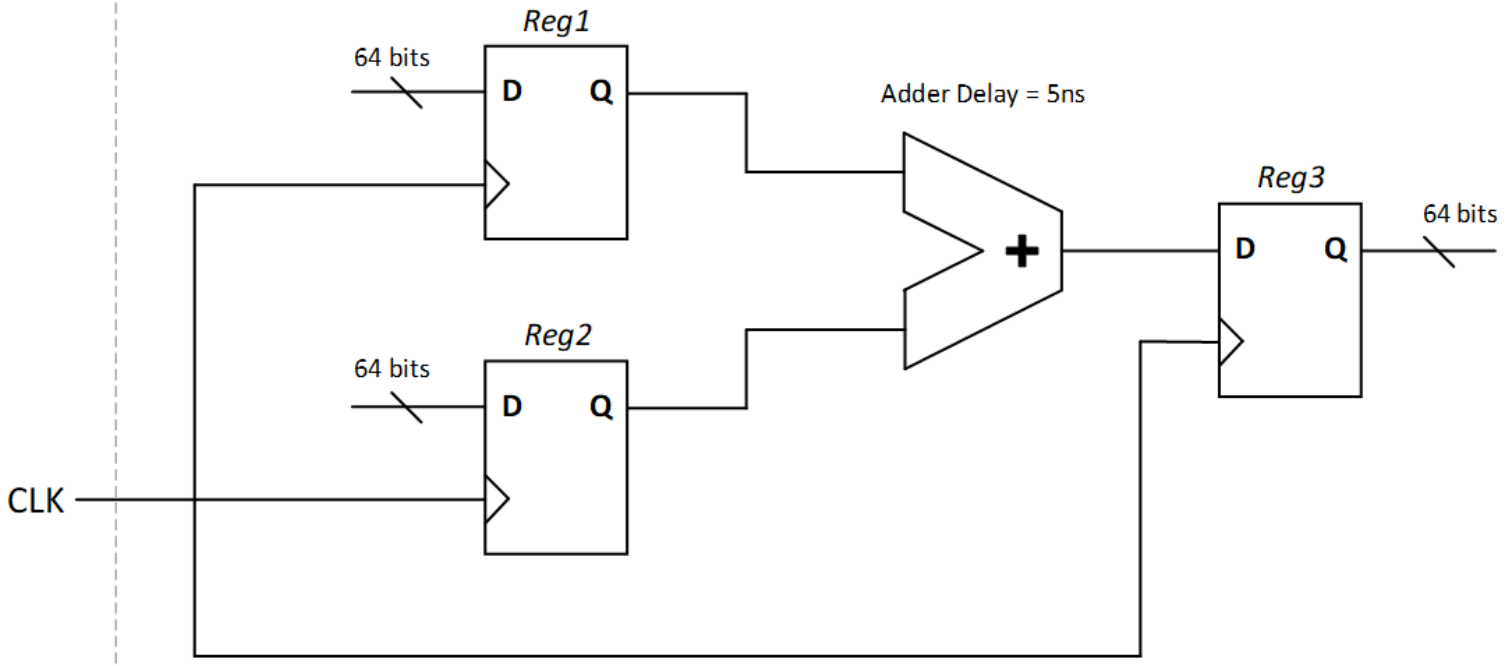
Static Timing Analysis

Multi-cycle Path:

- By defining the path as a multicycle path, you can tell the synthesis or STA tool that the path has 'N' clock cycles to propagate; so, the timing check becomes "the propagation .
- A Multi-Cycle Path (MCP) is a flop-to-flop path, where the combinational logic delay in between the flops is permissible to take more than one clock cycle. Sometimes timing paths with large delays are designed such that they are permitted multiple cycles to propagate from source to destination. Unlike false paths, multicycle paths are valid and must be analyzed, but against more than one clock period.

Static Timing Analysis

Multi-cycle Path:

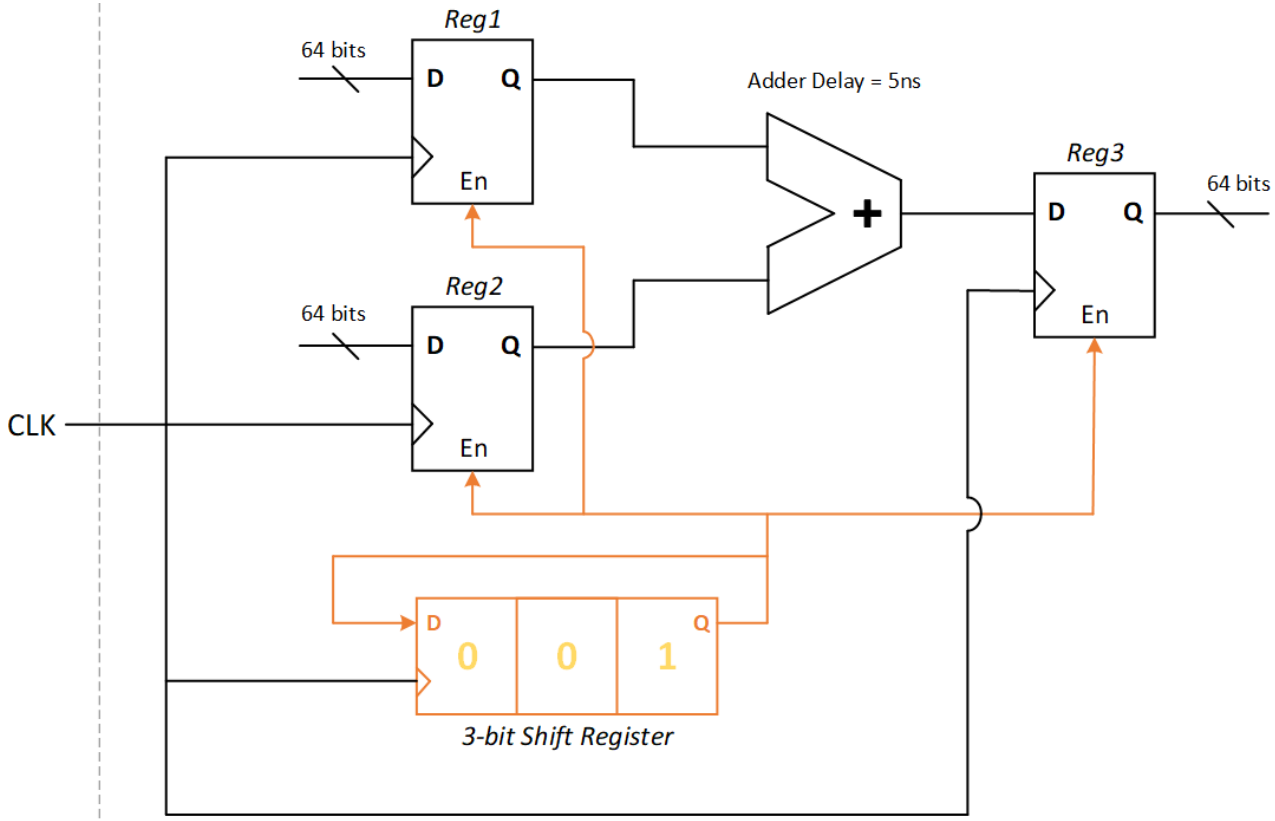


Consider an example as shown in this Figure, where we have only one clock (of 2ns period) in our design and we are performing a 64-bit addition of two buses. The input buses to the adder, as well as the output bus from the adder are registered. The maximum delay of the adder is estimated to be around 5ns, and the period of the register clock is 2ns. Since the adder delay is more than time period of the clock, it is not possible to close the timing within one clock cycle.

Static Timing Analysis

Multi-cycle Path:

For the circuit to operate correctly, a multi-cycle design approach was implemented. All the registers have a commonly connected enable bit, which is being controlled by a 3-bit shift register, which is preloaded with 001 and has a feedback loop from output to the input; this causes the enable signal to go high every 3 cycles.

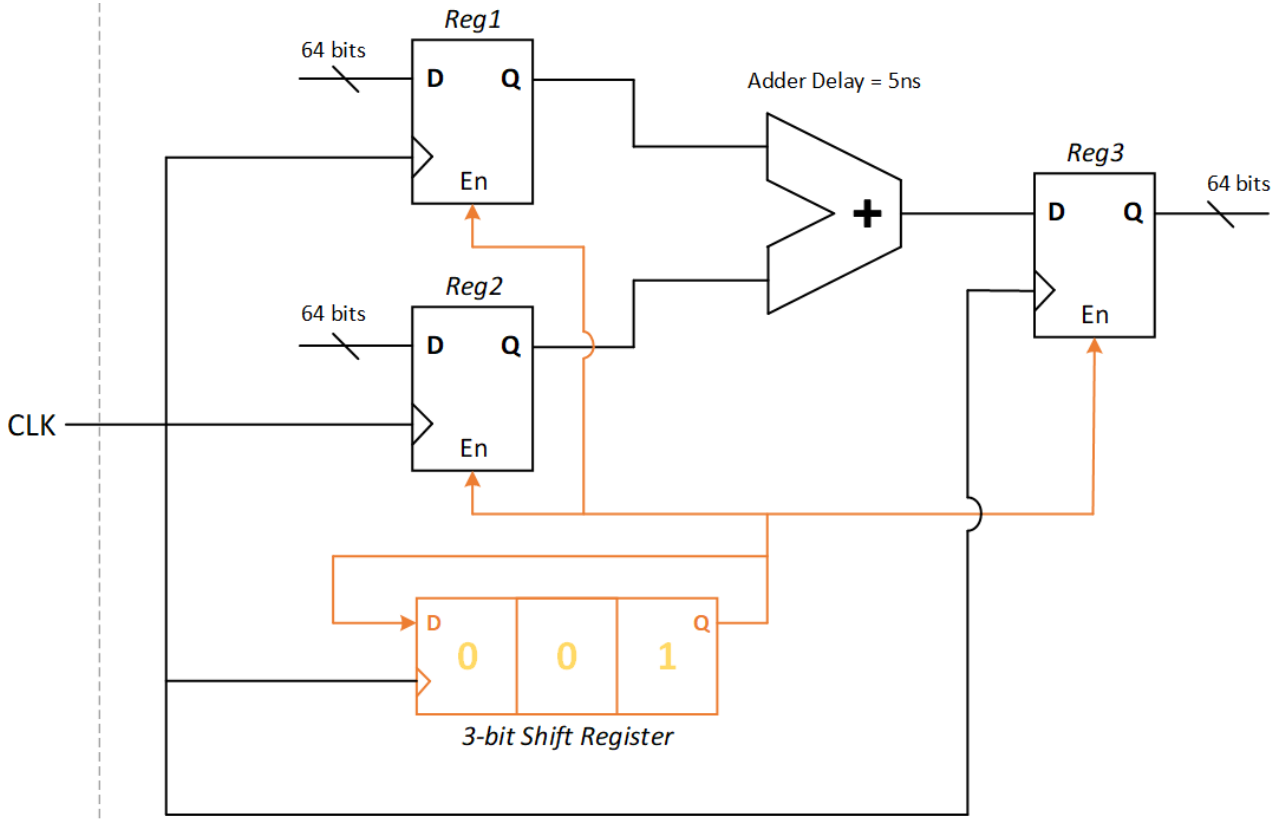


Static Timing Analysis

Multi-cycle Path:

If we simply apply a "*create_clock -period 2 [get_ports CLK]*" constraint, the synthesis tool will not analyze the "*En*" control logic to understand that the adder has 3 cycles for setup check timing closure (as the tool doesn't perform dynamic logic simulation).

∴ it will try to optimize the adder logic to close setup time within 1 clock cycle, which is not possible (and unnecessary).



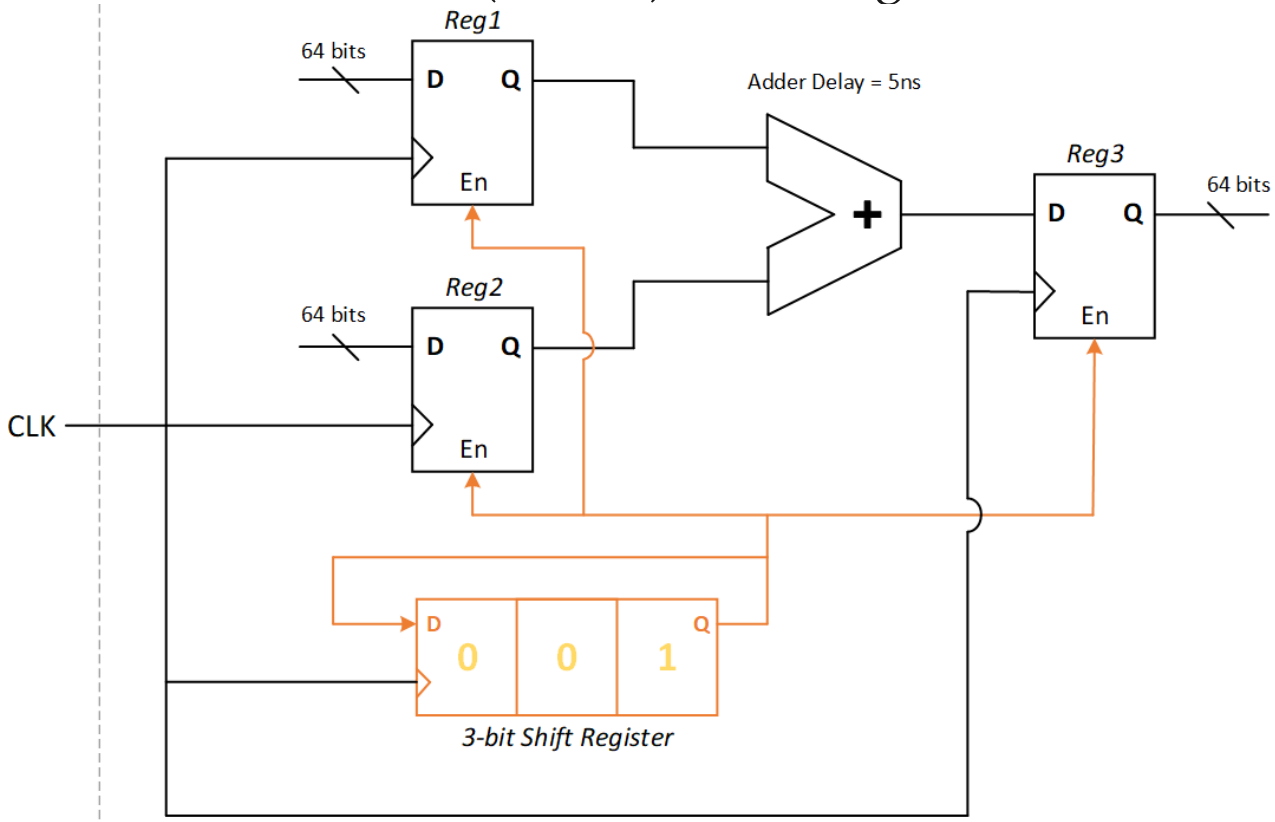
Static Timing Analysis

Multi-cycle Path:

∴ we need to add multi cycle paths between the input registers and the output register. This is done by the set_multicycle_path command and by specifying a setup value of 3 (corresponding to 3 cycles), which is then applied on the paths which start at Reg1 & Reg2 and end at Reg3, as shown –

set_multicycle_path -setup 3 -from {Reg1[*] Reg2[*]} -to Reg3[*]

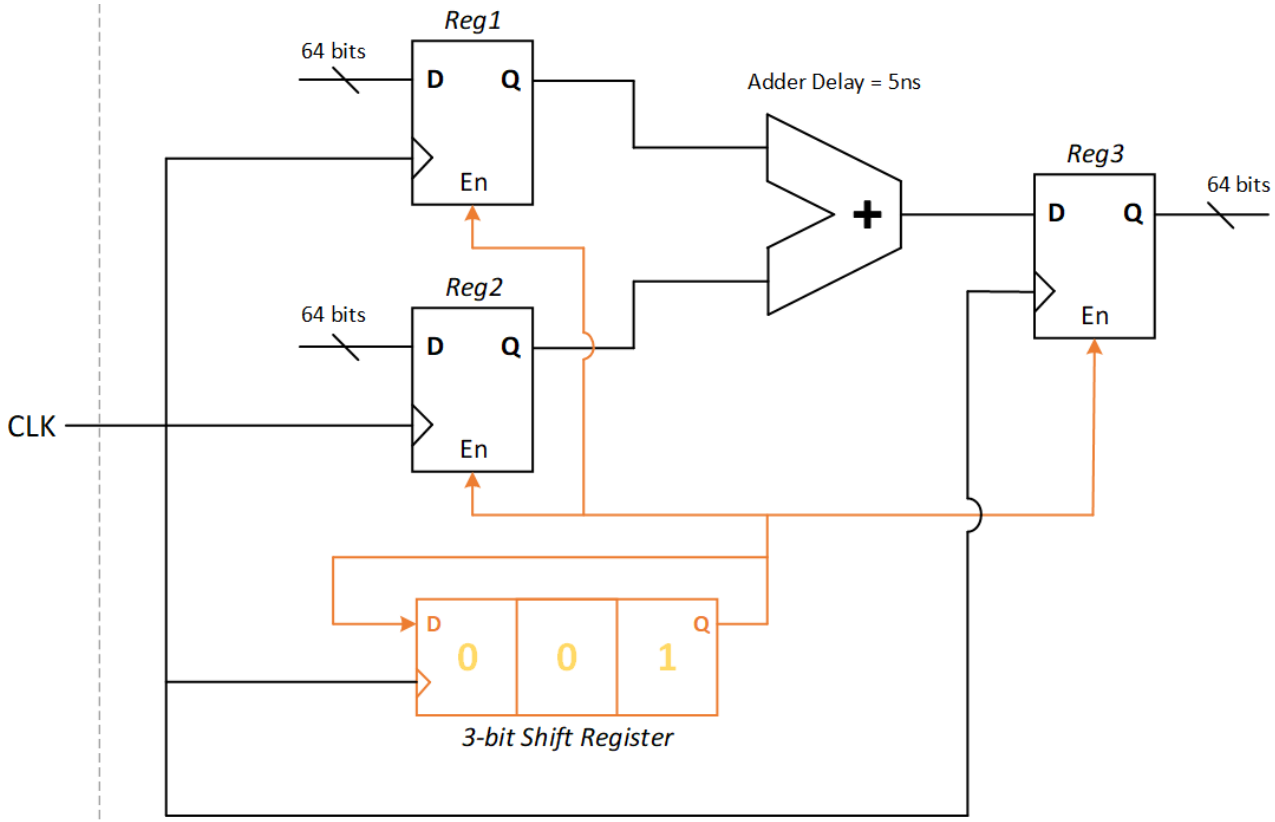
Note: The [*] is added to consider all the bits (64 bits) of the registers.



Static Timing Analysis

Multi-cycle Path:

By default (without any MCP constraint), setup checks are measured from the source clock edge to the destination's next clock edge and hold checks are measured from the source clock edge to the destination's same clock edge. In other words, we can also say hold timing analysis is performed in the same clock period where setup timing is performed.



Static Timing Analysis

Multi-cycle Path:

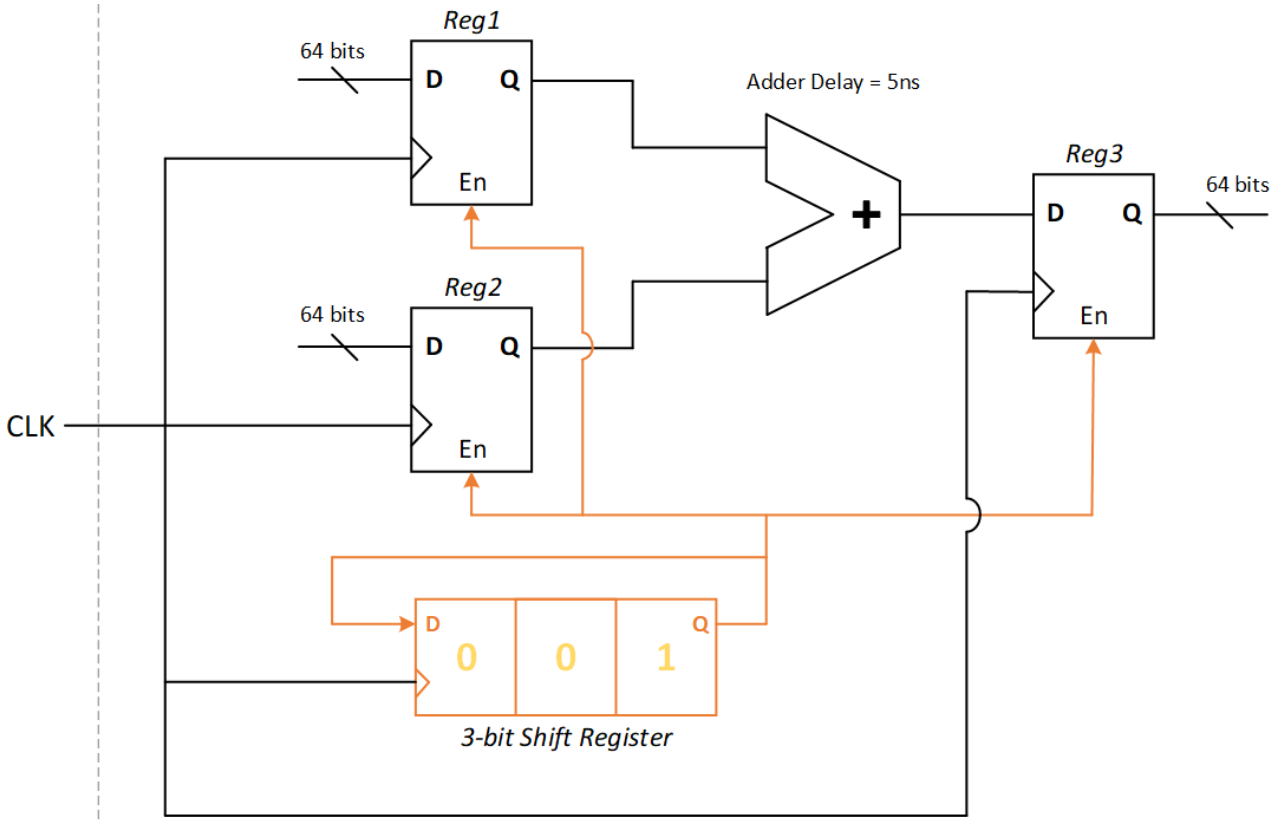
Let's say that a MCP setup of N was applied, if no explicit hold multi cycle path is set, the default hold check will be performed at $(N-1)^{th}$ edge of destination clock. In order to meet this hold requirement, additional buffers will be added by the synthesis tool, which un-necessarily increases area, power etc.

Typically, we only want to move the setup to the N^{th} edge of the destination clock and the hold check needs to be at the 0^{th} edge of destination clock. In order to do this, we need to specify both the setup and hold multi cycle path constraint.

Static Timing Analysis

Multi-cycle Path:

In the example we discussed, we have moved the setup capturing clock edge to the 3rd cycle (at 6ns), therefore the hold timing analysis is done by the synthesis tool at 4ns, which is within the same cycle as setup capturing edge. But this would mean the fastest path through this 64 bits adder under the fastest operating condition has to be slower than (4ns + hold time of Reg1 or Reg2), which is not necessary, as the registers are enabled registers, therefore we are not concerned about the possibility of metastability due to hold violation if the data happens to arrive right at one of the earlier clock edges between 0ns and 6ns.



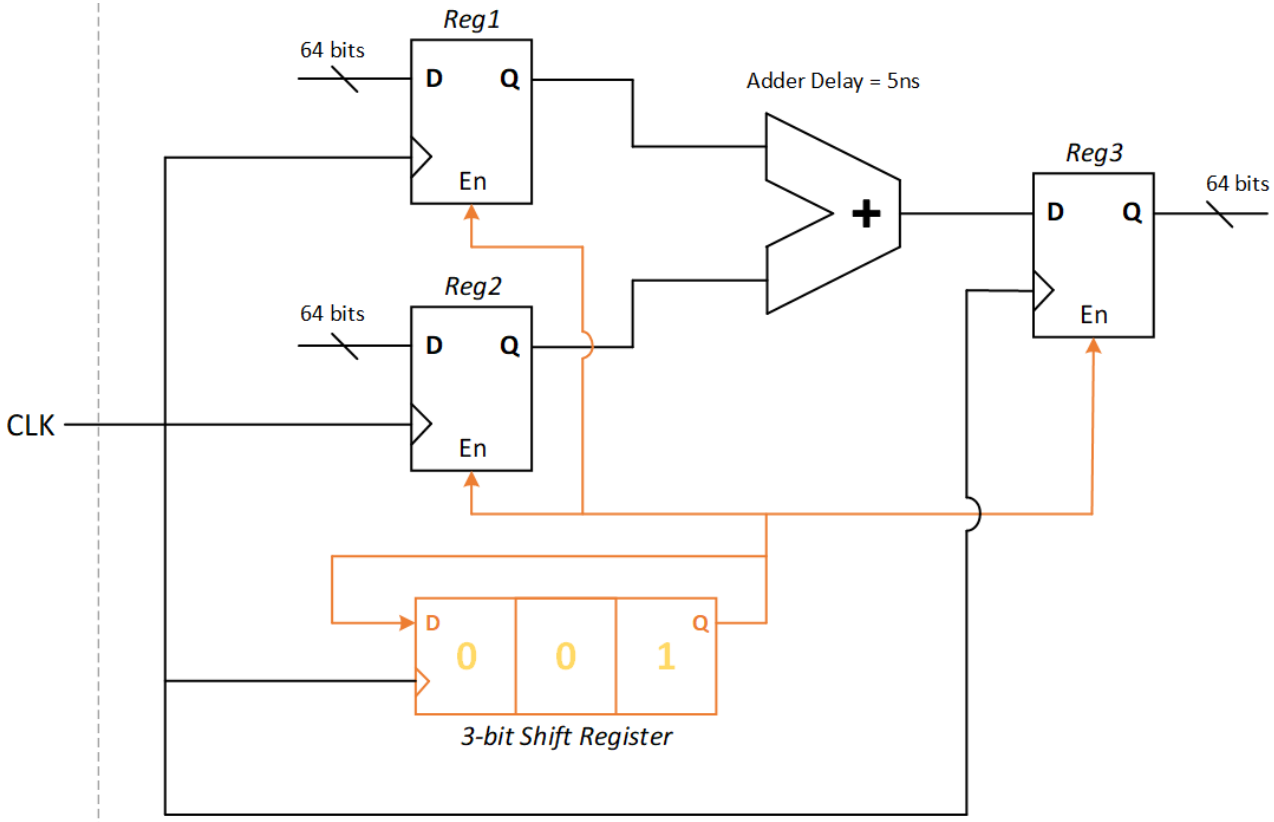
Static Timing Analysis

Multi-cycle Path:

So, we want the hold check to be performed at the same edge as the launching edge; to move the hold check back by 2 clock cycles, we need to use –

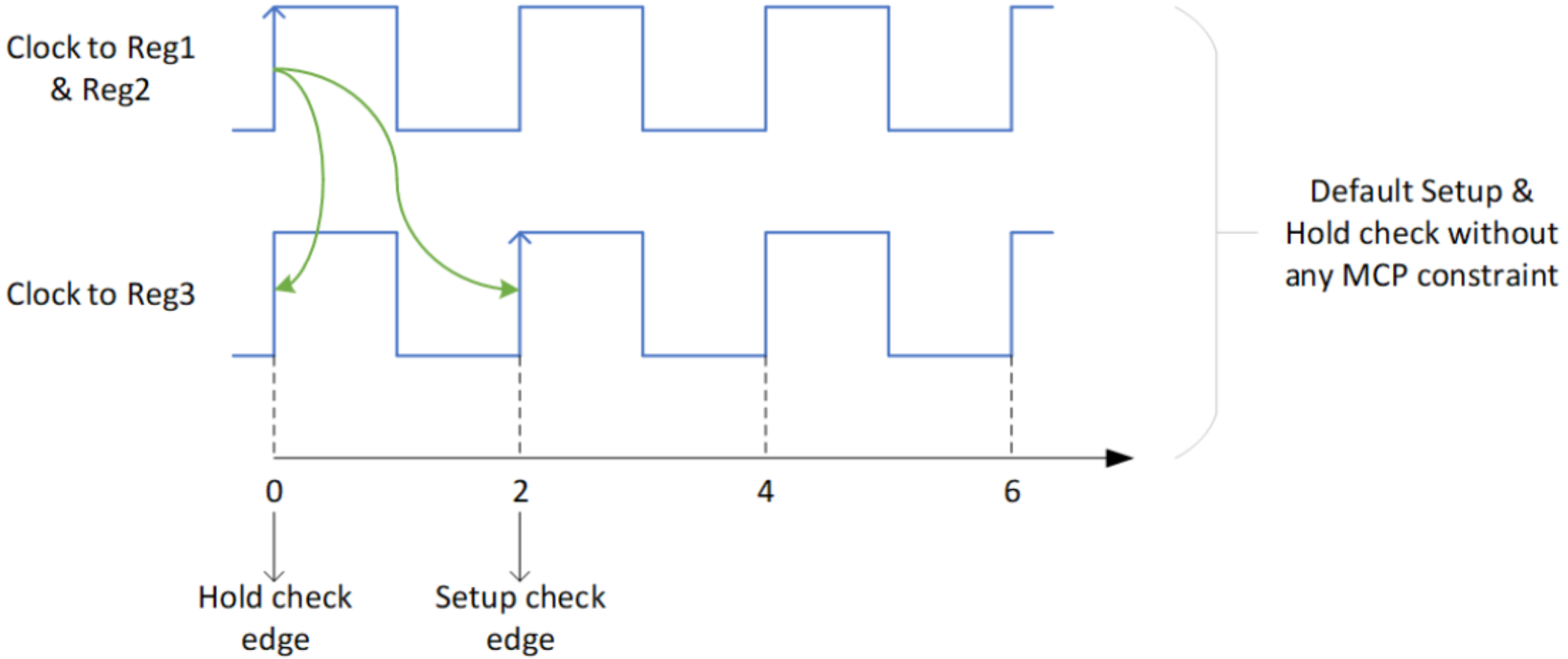
set_multicycle_path -hold 2 -from {Reg1[*] Reg2[*]} -to Reg3[*]

The default value of this hold option is 0, which means that the hold check is to be performed within the same cycle as the setup check, or in other words one clock cycle before the setup capturing clock edge happens. ∴ the guideline here is to specify both the setup and hold multi cycle path constraints, if we want the correct analysis by the timing engine tool.



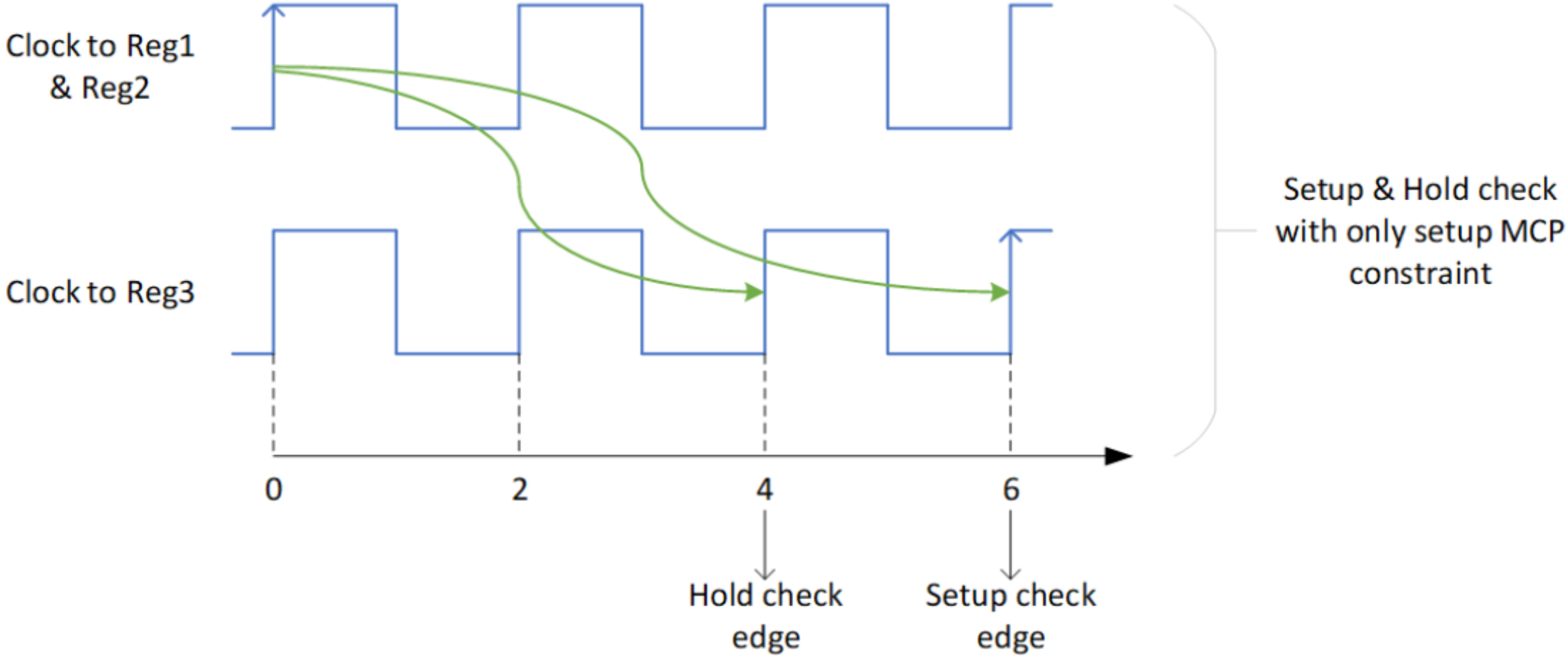
Static Timing Analysis

Multi-cycle Path:



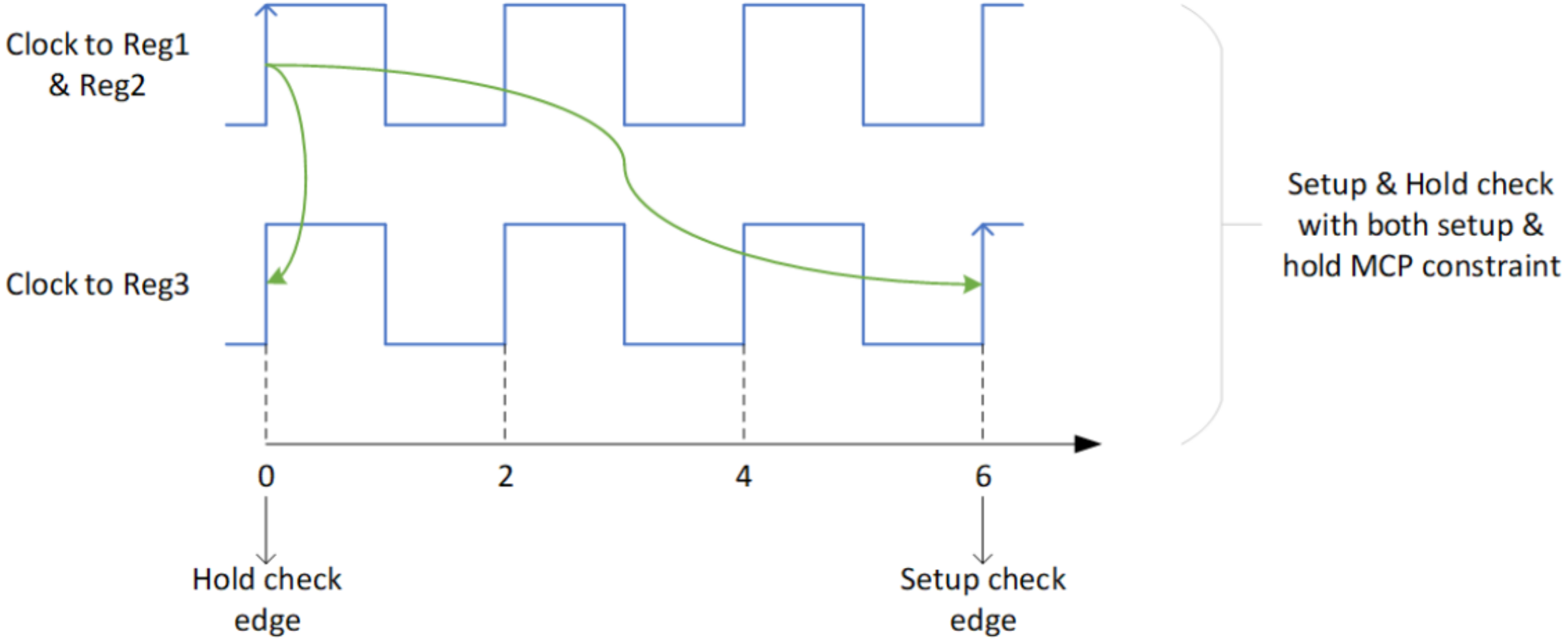
Static Timing Analysis

Multi-cycle Path:



Static Timing Analysis

Multi-cycle Path:



Static Timing Analysis

Numerical Problem:

Suppose a registered 64-bit adder operates with a clock of period 2 ns. The adder's delay is 5 ns, which is more than the clock period, so timing cannot close in a single cycle. Using a multi-cycle path constraint allows the data 3 cycles (6 ns) to traverse from input registers to the output register, matching the real architectural enable logic and making analysis correct.

Static Timing Analysis

Numerical Problem:

Suppose a registered 64-bit adder operates with a clock of period 2 ns. The adder's delay is 5 ns, which is more than the clock period, so timing cannot close in a single cycle. Using a multi-cycle path constraint allows the data 3 cycles (6 ns) to traverse from input registers to the output register, matching the real architectural enable logic and making analysis correct.

Problem Statement

- **Clock period:** 2 ns
- **Adder delay:** 5 ns
- **Path:** Reg1[] or Reg2[] → 64-bit adder → Reg3[] (where [] is all bits)
- **Registers:** Controlled by enable logic, enable goes high every 3 cycles

Without Multi-Cycle Path Constraint

- STA assumes data must be captured in 1 cycle.
- Setup check: Data must arrive at Reg3 (D) within 2 ns.
- Actual delay (5 ns) > clock period (2 ns): **Setup violation**.

Static Timing Analysis

Numerical Problem:

Suppose a registered 64-bit adder operates with a clock of period 2 ns. The adder's delay is 5 ns, which is more than the clock period, so timing cannot close in a single cycle. Using a multi-cycle path constraint allows the data 3 cycles (6 ns) to traverse from input registers to the output register, matching the real architectural enable logic and making analysis correct.

With Multi-Cycle Path Constraint

- Constraint: Data is valid after 3 cycles.
- SDC: *set_multicycle_path -setup 3 -from {Reg1[*] Reg2[*]} -to Reg3[*]*
- For setup, tool gives 3 × 2 ns = 6 ns for data to propagate.
- 5 ns < 6 ns: **No setup violation**.

Hold Check

- Default: STA checks hold at (N-1)th edge, i.e., after 2 cycles (4 ns).
- This isn't always desired; one typically wants the hold check at the launching edge (0 cycle).
- Correct with SDC: *set_multicycle_path -hold 2 -frm {Reg1[*] Reg2[*]} -to Reg3[*]*
- This aligns timing checks with architectural timing, avoiding unnecessary buffer insertion.

Calculation Table

Condition	Constraint	Allowed Time (ns)	Delay (ns)	Status
No MCP Constraint	1 cycle	2 ns	5 ns	Setup violation
MCP Constraint	3 cycles	6 ns	5 ns	Setup met

Static Timing Analysis

Numerical Problem:

Suppose a registered 64-bit adder operates with a clock of period 2 ns. The adder's delay is 5 ns, which is more than the clock period, so timing cannot close in a single cycle. Using a multi-cycle path constraint allows the data 3 cycles (6 ns) to traverse from input registers to the output register, matching the real architectural enable logic and making analysis correct.

With Multi-Cycle Path Constraint

- Constraint: Data is valid after 3 cycles.
- SDC: *set_multicycle_path -setup 3 -from {Reg1[*] Reg2[*]} -to Reg3[*]*
- For setup, tool gives 3 × 2 ns = 6 ns for data to propagate.
- 5 ns < 6 ns: **No setup violation**.

Hold Check

- Default: STA checks hold at (N-1)th edge, i.e., after 2 cycles (4 ns).
- This isn't always desired; one typically wants the hold check at the launching edge (0 cycle).
- Correct with SDC: *set_multicycle_path -hold 2 -from {Reg1[*] Reg2[*]} -to Reg3[*]*
- This aligns timing checks with architectural timing, avoiding unnecessary buffer insertion.

- **Multi-cycle path analysis** enables correct timing closure when data intentionally spans multiple clock cycles due to architectural enables or slow logic, letting the design use larger, slower logic and avoid unnecessary area/power increases.
- Both **setup and hold constraints** must be set for accurate STA interpretation and optimal synthesis, avoiding buffer bloat and missed hold checks.

Calculation Table

Condition	Constraint	Allowed Time (ns)	Delay (ns)	Status
No MCP Constraint	1 cycle	2 ns	5 ns	Setup violation
MCP Constraint	3 cycles	6 ns	5 ns	Setup met

Static Timing Analysis

Clock period (T): 4 ns
Combinational logic delay: 6 ns
Path: FF1 (launch register) → combinational logic → FF2 (capture register)
Registers FF1 and FF2 are on the same clock.
Required setup time for FF2: 0.2 ns

Static Timing Analysis

Clock period (T): 4 ns
Combinational logic delay: 6 ns
Path: FF1 (launch register) → combinational logic → FF2 (capture register)
Registers FF1 and FF2 are on the same clock.
Required setup time for FF2: 0.2 ns

Standard (1-cycle) STA Timing Check

- Default setup time budget: 1 × T = 4 ns
- Actual path delay = 6 ns (violates setup; timing cannot be met under default single-cycle assumptions.)

Static Timing Analysis

Clock period (T): 4 ns
Combinational logic delay: 6 ns
Path: FF1 (launch register) → combinational logic → FF2 (capture register)
Registers FF1 and FF2 are on the same clock.
Required setup time for FF2: 0.2 ns

Standard (1-cycle) STA Timing Check

- Default setup time budget: 1 × T = 4 ns
- Actual path delay = 6 ns (violates setup; timing cannot be met under default single-cycle assumptions.)

2-Cycle Multicycle Path: Calculations

Compute setup budget for 2-cycle path:

- Setup budget = 2 × 4 ns = 8 ns
- Adjusted for setup time: Allowed max path delay = 8 ns − 0.2 ns = 7.8 ns

Since 6 ns < 7.8 ns, the design meets setup for a 2-cycle multicycle path.

Static Timing Analysis

Clock period (T): 4 ns
Combinational logic delay: 6 ns
Path: FF1 (launch register) → combinational logic → FF2 (capture register)
Registers FF1 and FF2 are on the same clock.
Required setup time for FF2: 0.2 ns

Standard (1-cycle) STA Timing Check

- Default setup time budget: 1 × T = 4 ns
- Actual path delay = 6 ns (violates setup; timing cannot be met under default single-cycle assumptions.)

2-Cycle Multicycle Path: Calculations

Compute setup budget for 2-cycle path:

- Setup budget = 2 × 4 ns = 8 ns
- Adjusted for setup time: Allowed max path delay = 8 ns − 0.2 ns = 7.8 ns

Since 6 ns < 7.8 ns, the design meets setup for a 2-cycle multicycle path.

SDC Constraints for this example

To tell the STA tool that this path should be checked over 2 cycles rather than 1:

```
# Set setup check for 2 cycles
set_multicycle_path 2 -setup -from [get_pins FF1/Q] -to [get_pins FF2/D]

# Set hold check for 1 cycle (required for correct hold, as hold must move by (N-1))
set_multicycle_path 1 -hold -from [get_pins FF1/Q] -to [get_pins FF2/D]
```

Static Timing Analysis

Clock period (T): 4 ns
Combinational logic delay: 6 ns
Path: FF1 (launch register) → combinational logic → FF2 (capture register)
Registers FF1 and FF2 are on the same clock.
Required setup time for FF2: 0.2 ns

Standard (1-cycle) STA Timing Check

- Default setup time budget: 1 × T = 4 ns
- Actual path delay = 6 ns (violates setup; timing cannot be met under default single-cycle assumptions.)

2-Cycle Multicycle Path: Calculations

Compute setup budget for 2-cycle path:

- Setup budget = 2 × 4 ns = 8 ns
- Adjusted for setup time: Allowed max path delay = 8 ns − 0.2 ns = 7.8 ns

Since 6 ns < 7.8 ns, the design meets setup for a 2-cycle multicycle path.

The 2-cycle multicycle path increases the setup budget, ensuring timing is met for logic that topples a single-cycle window.

SDC Constraints for this example

To tell the STA tool that this path should be checked over 2 cycles rather than 1:

```
# Set setup check for 2 cycles
set_multicycle_path 2 -setup -from [get_pins FF1/Q] -to [get_pins FF2/D]

# Set hold check for 1 cycle (required for correct hold, as hold must move by (N-1))
set_multicycle_path 1 -hold -from [get_pins FF1/Q] -to [get_pins FF2/D]
```

Static Timing Analysis

Problem Statement

Clock period (T): 20 ns
FF1: Negative-edge triggered (data launched at 10 ns—falling edge)
FF2: Positive-edge triggered (data captured at 20 ns—rising edge)
Combinational delay (clk-Q + logic): 7 ns
Setup time (FF2): 1 ns

Static Timing Analysis

Calculation

Setup timing for half-cycle:

- Data is launched at 10 ns (falling edge) and must be captured at 20 ns (next rising edge).
- **Available time =** 20 ns – 10 ns = 10 ns (only half a cycle!)
- **Required:**$(\text{Launch edge time}) + \text{Path delay} \leq (\text{Capture edge time}) - \text{Setup}$
- $10\text{ns} + 7\text{ns} \leq 20\text{ns} - 1\text{ns} = 17\text{ns} \leq 19\text{ns}$
- **Result: No setup violation** (but path is tighter than normal 1-cycle, which would have allowed 20 ns).

Hold timing for half-cycle:

- Hold is checked at "previous" capture edge: 0 ns (previous rising edge).
- Data must arrive after 0 ns: $10\text{ns} + 7\text{ns} \geq 0\text{ns} + \text{Hold} = 17\text{ns} \geq \text{Hold}$

Hold is almost always easy in half-cycle paths.

Key takeaway: In a half-cycle path, setup time is checked across only half the clock period, making the timing check more stringent.

Static Timing Analysis

False Path: Scenario

- Consider two flops connected through a control/status register path used only during a special mode—never in normal operation data flow.
- Clock period (T): 10 ns
- Path delay: 15 ns
- **This path is NOT required to meet single-cycle timing during normal operation.**

STA Treatment

- With no constraints, STA will report a violation because 15 ns > 10 ns.
- However, **designer knows this path is irrelevant for system performance** (e.g., only used during slow configuration on chip reset).
- **Mark the path as false:**

```
set_false_path -from [get_pins FF1/Q] -to [get_pins FF2/D]
```

- Once marked, STA ignores this path for timing analysis.

Result: No violation reported, tools focus on timing closure for actual data paths only.

Key takeaway: False paths are intentionally ignored by STA using `set_false_path` constraint—they are not analyzed for timing, as their function does not impact system behavior.

Introduction to Clock Skew

- **Clock skew**: The difference in arrival time of the clock signal at two sequential elements (registers/flip-flops).
- **Types:**
  - **Positive skew**: Capture clock delayed; helps setup, hurts hold.
  - **Negative skew**: Capture clock earlier; hurts setup, helps hold.

Main causes: Wire length, buffer insertion, process variations, duty cycle distortion.

Introduction to Clock Skew

- **Clock skew**: The difference in arrival time of the clock signal at two sequential elements (registers/flip-flops).
- **Types:**
  - **Positive skew**: Capture clock delayed; helps setup, hurts hold.
  - **Negative skew**: Capture clock earlier; hurts setup, helps hold.

Main causes: Wire length, buffer insertion, process variations, duty cycle distortion.

Impact of Clock Skew

- **Setup check:** $T_{CQ}+T_{COMB} + T_{SU} < T_{CLK} + T_{skew}$
- **Hold check:** $T_{CQ} + T_{COMB} > T_{skew} + T_{hold}$

Useful Skew: Sometimes, controlled skew is introduced to manage setup/hold trade-offs.

Techniques for Clock Skew Optimization

1. Balanced clock tree synthesis (CTS)

- Build H-tree or X-tree clock distribution.
- Ensure buffer/load symmetry, minimize interconnect variation.

2. Buffer insertion/sizing

- Add or size clock tree buffers for delay balancing.
- Tool-assisted or hand-optimized.

3. Wire length and routing

- Equalize clock path lengths from source to sinks.
- Avoid unnecessary detours and branching.

4. Useful clock skew

- Introduce deliberate skew where timing margins permit.
- Managed at the STA/constraint level.

Techniques for Clock Skew Optimization

Balanced clock tree synthesis (CTS)

- Aims to distribute the clock signal uniformly so all flip-flops receive the clock almost simultaneously.
- Uses structures like H-tree and X-tree for symmetry, which minimizes skew.
- Strategic buffer and inverter placement corrects small delays and ensures load balancing.

Implementation/Verification (Verilog):

Balancing is performed physically via EDA synthesis; simulation can represent ideal or delayed clocks:

```
// Simulate two balanced clock branches
reg clk, clk_branch1, clk_branch2;

initial begin
  clk = 0;
  forever #5 clk = ~clk; // 10ns period
end

always @(clk) begin
  // Balanced branches: same delay
  #1 clk_branch1 = clk; // 1ns buffer delay
  #1 clk_branch2 = clk; // 1ns buffer delay
end
```

This shows equal delays to both branches, imitating clock tree balancing.

Techniques for Clock Skew Optimization

Buffer Insertion/Sizing

- Buffers are added or sized within the clock paths to manage delay and minimize skew for distant sinks.
- Buffer levels are matched across branches, and sizing is tool-optimized or can be hand-tuned.

Implementation/Verification (Verilog):

```
// Instantiate clock buffers with specific delays
wire clk_buf1, clk_buf2;
buf #(2) u_buf1(clk_buf1, clk); // 2ns delay buffer
buf #(4) u_buf2(clk_buf2, clk); // 4ns delay buffer

always @(posedge clk_buf1) begin
  // register logic for branch 1
end

always @(posedge clk_buf2) begin
  // register logic for branch 2
End
```

Here, changing buffer delay values models different path skews; physical design would optimize these in layout.

Techniques for Clock Skew Optimization

Wire Length and Routing

- Routing tools aim to equalize wire lengths from clock source to sinks to reduce skew.
- Avoids unnecessary turns and splits; strategic placement groups minimize clock detours.

Implementation/Verification (Verilog):

Software simulation cannot directly alter wire length, but delay modeling achieves the same effect:

```
// Route simulation with variable net delay
wire clk_long, clk_short;
assign #3 clk_long = clk;   // 3ns net delay simulates a longer wire
assign #1 clk_short = clk;  // 1ns net delay simulates a shorter wire

always @(posedge clk_long)
  // logic for distant registers

always @(posedge clk_short)
  // logic for close registers
```

Synthesis tools would re-route the clock nets to achieve symmetry and minimal skew.

Techniques for Clock Skew Optimization

Useful Clock Skew

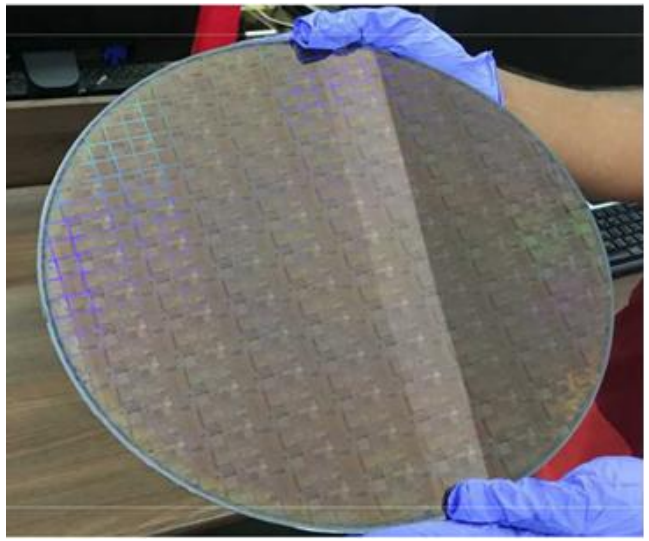
- Sometimes designers intentionally introduce skew if timing margins permit, shifting capture edges for setup or hold improvement.
- Managed by STA tools via constraints (SDC) or explicit clock phase adjustments.

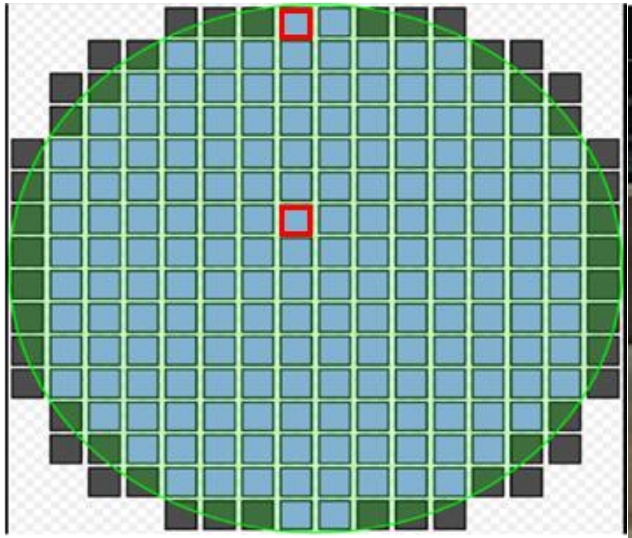
Implementation/Verification (Verilog):

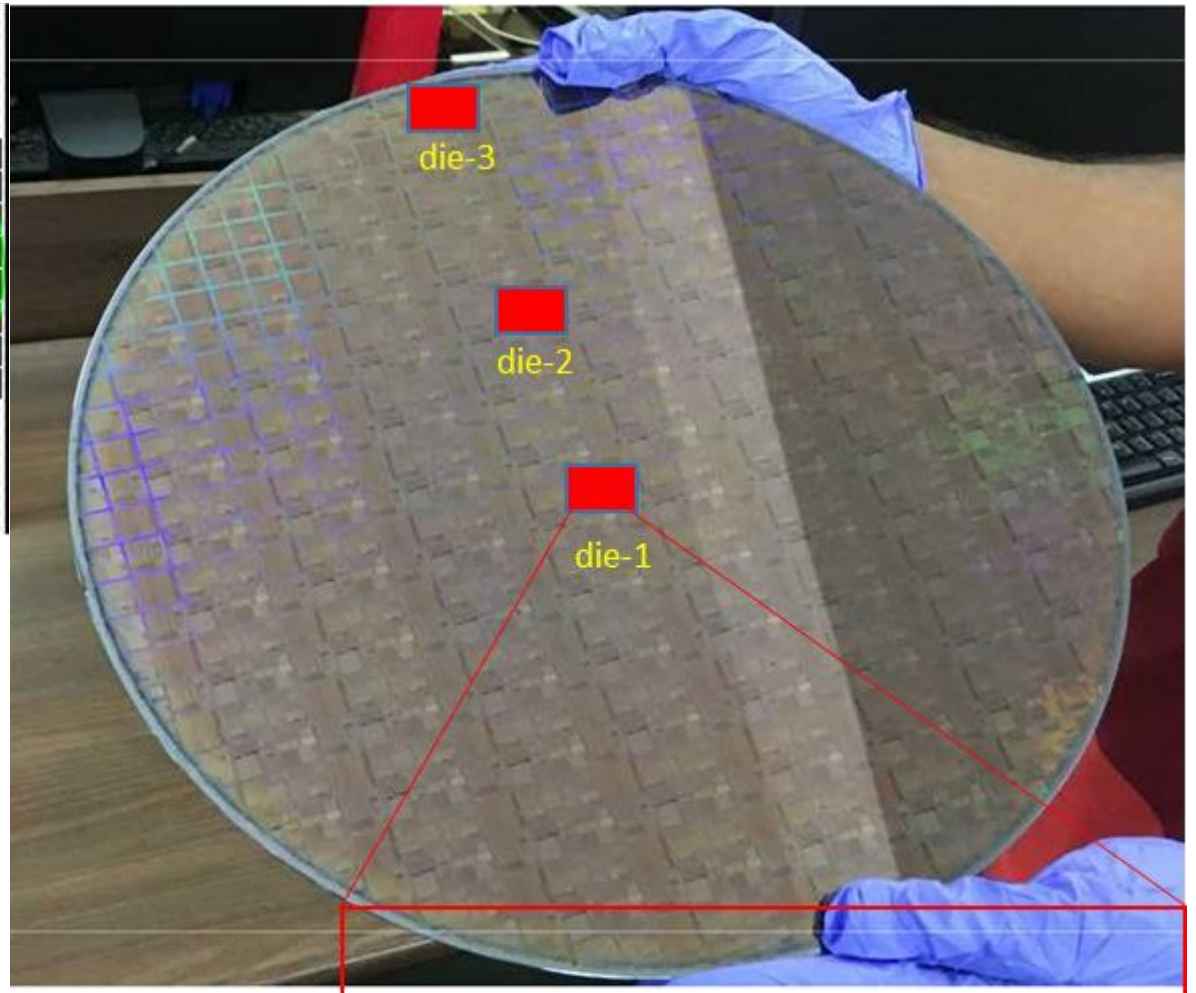
```
// Simulate useful skew by offsetting capture clock
reg clk_launch, clk_capture;

initial begin
  clk_launch = 0; clk_capture = 0;
  forever #10 clk_launch = ~clk_launch;        // Launch clock (20ns period)
end

always @(clk_launch)
  #2 clk_capture = clk_launch; // Capture clock delayed by 2ns (useful skew)

always @(posedge clk_launch) begin
  // Launch data
end

always @(posedge clk_capture) begin
  // Capture data with skewed clock
end
```

In STA, we use: `set_clock_latency -source 2.0 [get_clocks clk_capture]`

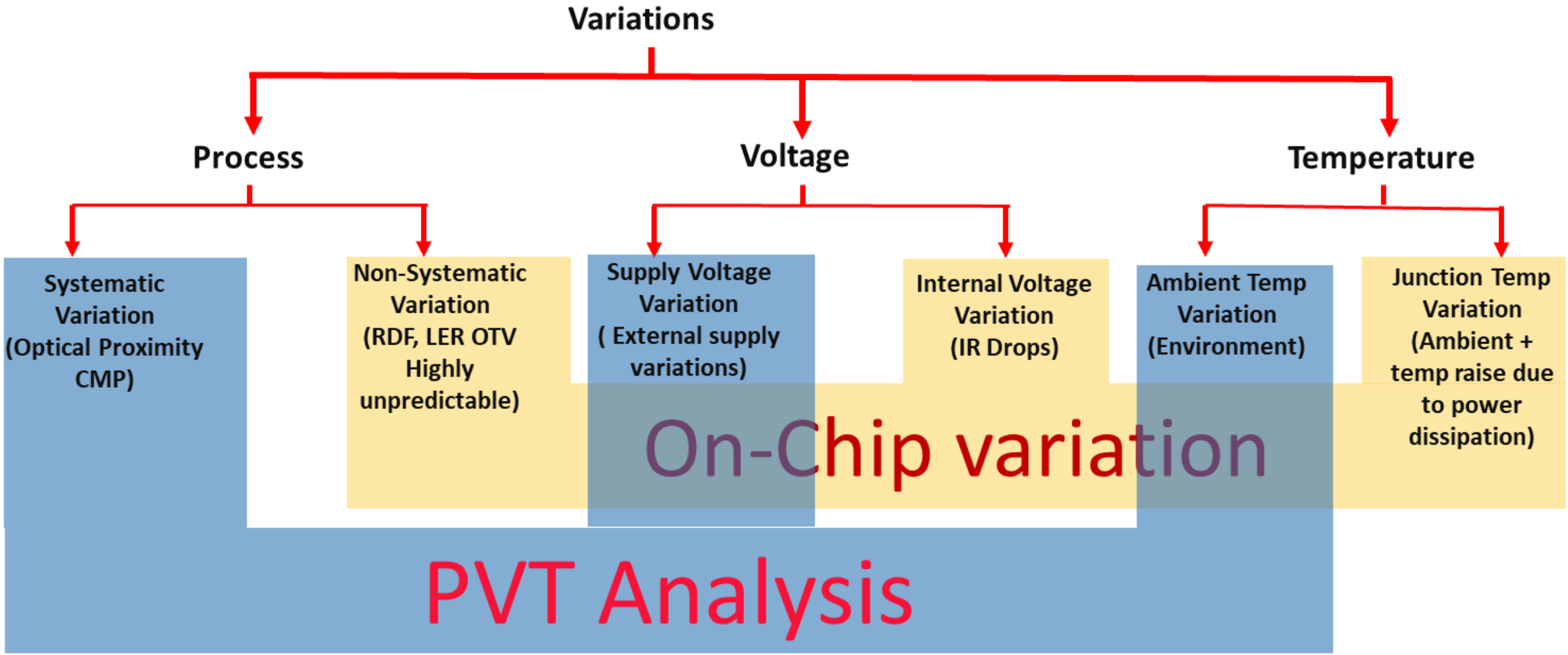
On-Chip Variation (OCV): Fundamentals







On-Chip Variation (OCV): Fundamentals



On-Chip Variation (OCV): Fundamentals

How to take care of OCV:

To take care of OCV we need to add some pessimism in the timing of standard cells. We basically apply ±x% of additional delay to all the standard cells. Which is called OCV derate.

OCV derate factor:

Derate factor is a very simple approach to take of on chip variation. A fixed derate factor is applied on throughout the design. So that in case of any variation occurs will not cause the failure of the chip. But it added too much of timing pessimism which leads to difficulties in the timing closure, especially in the lower nodes.

So, the industry has moved to different concepts from the fixed derate to distance and depth based derate which is called Advance On Chip Variation (AOCV). As the technology node further shrank more, AOCV also is not a good option and further Parametric On Chip Variation (POCV) evolved. We will discuss OCV and AOCV in subsequent slides. In short, it can be proclaimed that as the industry has moved from OCV to POCV, the timing pessimism is reduced. POCV is not in syllabus of BECE407E.

On-Chip Variation (OCV): Fundamentals

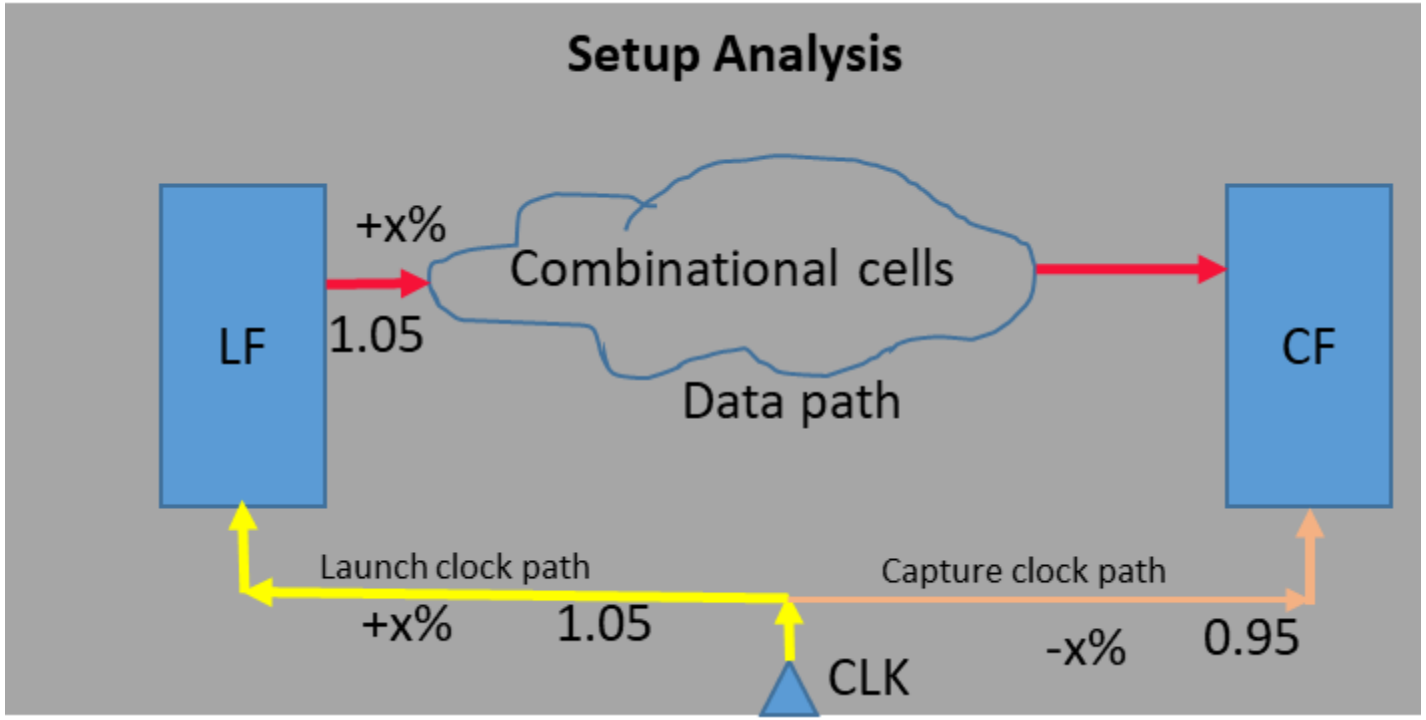
- In OCV a fixed timing derate factor is applied to the delay of all the cells present in the design so that in case of process variation affects the delay of any cells during the fabrication, it will not affect the timing requirements and the chip will not fail after fabrication.
- Fabrication process variations could either increase or decrease the delay of a cell. So, we need to set early and late values while setting the derate factor.
- STA tool would consider early or late timing derate based on the path and type of analysis.

- Here is an example of setting the OCV timing to derate factor.

```
set_timing_derate -cell_delay-rise -data -early 0.92
set_timing_derate -cell_delay-rise -data -late 1.10
set_timing_derate -cell_delay-rise -clock -early 0.95
set_timing_derate -cell_delay-rise -clock -late 1.06
set_timing_derate -cell_delay-fall -data -early 0.90
set_timing_derate -cell_delay-fall -data -late 1.12
set_timing_derate -cell_delay-fall -clock -early 0.94
set_timing_derate -cell_delay-fall -clock -late 1.07
```

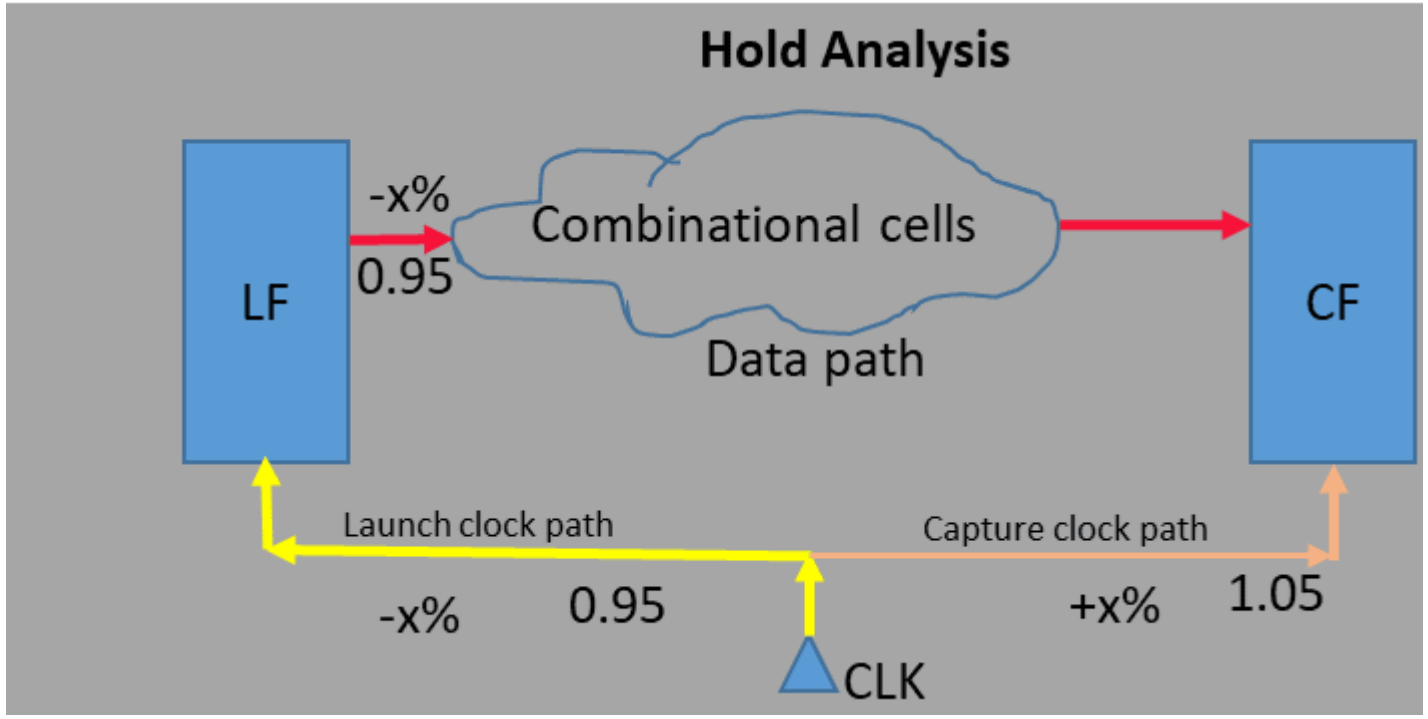
- In the above example, line-1 sets 8% early timing derate and line-2 sets 10% late timing derate to the rising edge on the data path. Similarly, line-4 and line-5 sets 5% early and 6% late timing derate to the rising edge on the clock path.

OCV: Derate factor on setup analysis



- In reg2reg path timing analysis there is a launch flop from where data is launched and a capture flop where data is captured. The path between the clock source to the clock pin of the launch flop is called the launch clock path and the path from the clock source to the clock pin of the capture flop is called the capture clock path.
- In setup analysis worse case could be a late data path, late launch clock path and early capture clock path which could fail the setup timing. So, STA tool will consider late timing derate for data path and launch clock path and early derate for the capture clock path.

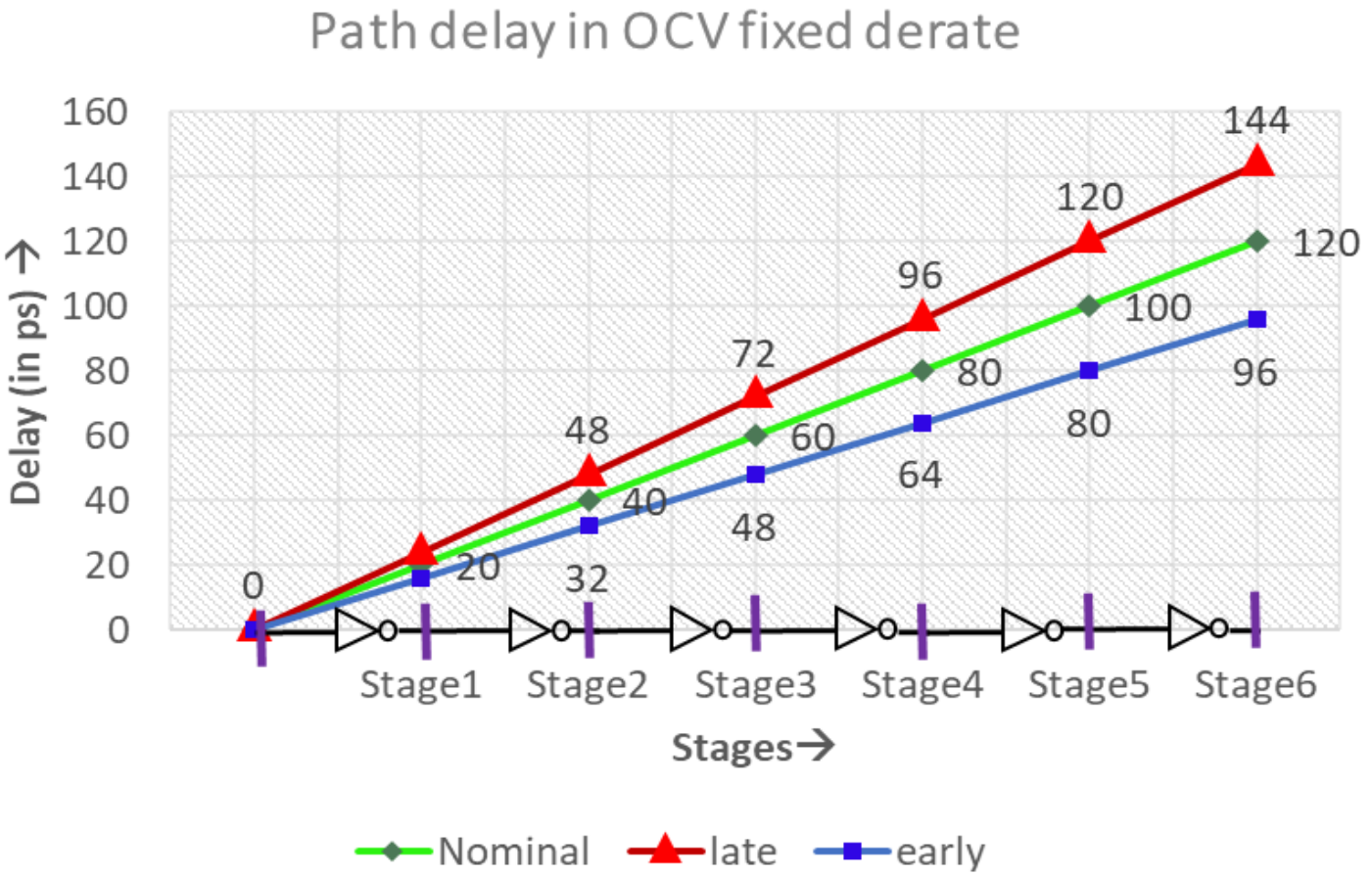
OCV: Derate factor on hold analysis



- In reg2reg path timing analysis there is a launch flop from where data is launched and a capture flop where data is captured. The path between the clock source to the clock pin of the launch flop is called the launch clock path and the path from the clock source to the clock pin of the capture flop is called the capture clock path.
- For hold analysis, a fast data path, early launch clock and late capture clock could be the worst scenario. So, STA tool will consider always the worst scenario and take the early derate factor for the data path and launch clock path and the late derate factor for capture clock path.

Issues of OCV

- Fixed timing derate used for all the cells in the OCV is over pessimistic.
- In reality, there is the cancellation of random variation effect.
- All the cells in a particular path could not be delayed all or early all.
- There is a mixed type of effect always and this causes cancellation of effect in total.



For example, consider a data path of 6 buffers and their typical delay is 20ps of each cell. Consider 20% late an early derate. So, considering all the cells have the effect of late in delay, this path will have maximum delay 144ps. But in practice, it is very rare that all the cell will have an effect either late only or early only. Most likely some will be late and some will be early so there will be a cancellation of effect and real delay will always be less than 144ps.

Issues of OCV

The concepts of OCV fixed derate was modelled in technology node above 90nm. It was good for such higher technology node. But in lower technology node and especially high-frequency designs, the timing closer became very difficult due to the high pessimism of fixed derate. So, for the lower technology node, we need to resolve this issue. And so, the concept of Advance On Chip Variation (AOCV) has evolved which does not use the fixed derate.

On-Chip Variation (OCV): Fundamentals

OCV refers to the timing variability caused by process, voltage, and temperature (PVT) fluctuations across a chip, affecting gate and path delays unpredictably.

OCV is modeled in timing analysis by applying +/- percentage derates to cell delays (e.g., +8% for launch, -9% for capture) to simulate worst-case scenarios in STA.

Example: For a path with a nominal delay of 100ps, OCV considers launch = 108ps, capture = 91ps for setup analysis.

OCV Timing Derate: Step-by-Step Example

1. Base Scenario (No OCV)

- Setup scenario: Data path from FF1 to FF2
  - Clock period: 1000 ps
  - Data path delay: 100 ps (nominal)
  - Clock path delay: 50 ps (nominal)
  - Setup time (FF2): 10 ps
  - Required time at FF2 for Data flow: 1000 ps - 50 ps - 10 ps = 940 ps
  - Slack: 940 ps - 100 ps = 840 ps

2. OCV Derate Applied

- Assumptions: OCV derate for setup
  - Launch (data) path: +8%
  - Capture (clock) path: -9%
- Calculation:
  - Data path effective delay: 100 ps × 1.08 =108 ps
  - Clock path effective delay: 50 ps × 0.91 = 45.5 ps
  - Required time at FF2 for Data flow: 1000 ps - 45.5 ps - 10 ps = 944.5 ps
  - Slack: 944.5 ps - 108 ps = 836.5 ps

OCV Timing Derate: Step-by-Step Example

3. PrimeTime SDC Command Example

```
# Set OCV derate in Synopsys SDC
set_timing_derate -late 1.08 -data_pins; # Data path, apply late derate for setup
set_timing_derate -late 0.91 -clock_pins; # Clock path, apply capture derate for setup
```

These commands make the timing analyzer use the scaled values above for setup analysis.

4. Generalized Setup Timing Equation with OCV

$$T_{clk} > (T_{cq,launch} + T_{data,nom} \times D_{launch}) + T_{setup} - (T_{clk,capture} + T_{clk,nom} \times D_{capture})$$

where:

$T_{data,nom}$ is nominal data path delay.
D_{launch} is launch derate (e.g. 1.08).
$D_{capture}$ is capture derate (e.g. 0.91).

OCV Timing Derate: Step-by-Step Example

5. PrimeTime SDC Command Example

```
set_timing_derate -early 0.91 -data_pins; # Data path early derate for hold
set_timing_derate -late 1.08 -clock_pins; # Clock path late derate for hold
```

This ensures the STA tool applies pessimistic values for the worst-case hold analysis.

6. Generalized Hold Timing Equation with OCV

The formal hold timing check (without OCV) is:

$$\text{Data Arrival Time} - \text{Data Required Time} \geq 0$$

For a launch flip-flop (FF1) to capture flip-flop (FF2) path, and considering OCV:

$$\left(T_{cq,launch} + T_{data,nom} \times D_{launch}^{hold}\right) - \left[T_{clk,capture} + T_{clk,nom} \times D_{capture}^{hold} + T_{hold}\right] \geq 0$$

where:

$T_{cq,launch}$ = clock-to-Q delay of launch FF (not derated)
$T_{data,nom}$ = nominal data path delay
D_{launch}^{hold} = launch derate for hold (early/fast, e.g., 0.91 for −9%)
$T_{clk,capture}$ = clock network delay to capture FF (nominal)
$T_{clk,nom}$ = nominal capture clock path delay
$D_{capture}^{hold}$ = capture derate for hold (late/slow, e.g., 1.08 for +8%)
T_{hold} = hold time of capture FF

OCV Timing Derate: Step-by-Step Example

5. PrimeTime SDC Command Example

```
set_timing_derate -early 0.91 -data_pins; # Data path early derate for hold
set_timing_derate -late 1.08 -clock_pins; # Clock path late derate for hold
```

This ensures the STA tool applies pessimistic values for the worst-case hold analysis.

6. Generalized Hold Timing Equation with OCV

The formal hold timing check (without OCV) is:

$$\text{Data Arrival Time} - \text{Data Required Time} \geq 0$$

For a launch flip-flop (FF1) to capture flip-flop (FF2) path, and considering OCV:

$$\left(T_{cq,launch} + T_{data,nom} \times D_{launch}^{hold}\right) - \left[T_{clk,capture} + T_{clk,nom} \times D_{capture}^{hold} + T_{hold}\right] \geq 0$$

where:

$T_{cq,launch}$ = clock-to-Q delay of launch FF (not derated)
$T_{data,nom}$ = nominal data path delay
D_{launch}^{hold} = launch derate for hold (early/fast, e.g., 0.91 for −9%)
$T_{clk,capture}$ = clock network delay to capture FF (nominal)
$T_{clk,nom}$ = nominal capture clock path delay
$D_{capture}^{hold}$ = capture derate for hold (late/slow, e.g., 1.08 for +8%)
T_{hold} = hold time of capture FF

Interpretation:
Launch path uses the minimum (fastest) possible delay → early derate.
Capture clock path uses the maximum (slowest) possible delay → late derate.

OCV Timing Derate: Step-by-Step Example

Numerical Example

Suppose:

- Data path nominal: 100ps, derate 0.91 ⇒ 91ps (fast)
- Capture clock path nominal: 50ps, derate 1.08 ⇒ 54ps (slow)
- T_{hold}= 10ps

$$\text{Hold Slack} = (91\text{ps}) - (54\text{ps} + 10\text{ps}) = 91 - 64 = 27\text{ps}$$

If result is negative, a hold violation occurs.

Advanced OCV (AOCV): Depth & Distance Aware Modeling

Advanced OCV (AOCV) introduces a more specific derating model using two dimensions: timing path depth (*number of logic stages*) and physical distance (*cell group separation*) to refine variability margins. This targets less pessimism and improved accuracy versus fixed OCV.

Advanced OCV (AOCV): Depth & Distance Aware Modeling

Advanced OCV (AOCV) introduces a more specific derating model using two dimensions: timing path depth (*number of logic stages*) and physical distance (*cell group separation*) to refine variability margins. This targets less pessimism and improved accuracy versus fixed OCV.
In AOCV, *derate is applied on each cell based on path depth and distance of the cell in the timing path* and it also *varies with cell type and drive strength of the cell*.

Advanced OCV (AOCV): Depth & Distance Aware Modeling

Distance: If the distance increases, systematic variation would increase and to mitigate the variation, we need to use higher derate value. So along with the distance, derate value increases.
Path depth: In the case of distance is fixed and path depth increases, systematic variation would be constant but the random variation would tend to cancel each other. Therefore, as path depth increases the derate factor would decrease.

The derate factor D is generally expressed as a function:

$$D = f(depth, distance, cell\ type)$$

where:

- **depth** corresponds to the number of logic levels in the path, capturing the effect of random variation cancellation as depth increases.
- **distance** refers to the physical distance between the cells on the chip, capturing systematic variation that grows with distance.
- **cell type** accounts for different variation sensitivities for different types and drive strengths of cells.

Advanced OCV (AOCV): Depth & Distance Aware Modeling

A simplified equation form used in analysis for a given cell delay derate could be:

$$\textbf{Derated delay} = \textit{Nominal delay} \times \textit{DepthDerate}(\boldsymbol{d}) \times \textit{DistanceDerate}(\boldsymbol{s})$$

where:

- **DepthDerate**($\boldsymbol{d}$) decreases with depth d, reflecting cancellation of random variation over multiple stages.
- **DistanceDerate**($\boldsymbol{s}$) increases with the distance s, addressing systematic variations increasing with physical separation.

In handling the total **derate factor**:

•For cells with AOCV models:

$$\textbf{Total Derate} = \textit{AOCV Derate} \times \textit{Guardband Scale} + \textit{Additive Margin}$$

•For cells without AOCV models:

$$\textbf{Total Derate} = \textit{Flat Scaling} + \textit{Additive Margin}$$

This equation incorporates both multiplicative and additive derate components for timing margining.

AOCV Numerical Examples

Problem 1: Depth and Distance Based Derate Calculation

Given:

- Nominal cell delay = 1ns
- Path depth (number of stages) = 6
- Physical distance between cells on the path = 500µm
- Depth Derate factor at depth=6 = 0.85 (reflecting random variation cancellation)
- Distance Derate factor at 500µm = 1.10 (reflecting increased systematic variation)
- Guardband scale = 1.05
- Additive margin = 0.02ns

Calculate:

- Derated delay for a cell with AOCV model.
- Total derate factor and adjusted delay applying guardband and margin.

AOCV Numerical Examples

Solution:

$$\text{Derated delay} = \textit{Nominal delay} \times \textit{DepthDerate}(6) \times \textit{DistanceDerate}(500)$$
$$= 1 \times 0.85 \times 1.10 = 0.935ns$$
$$\text{Total Derate} = \left(\textit{Derated delay} \times \textit{Guardband Scale}\right) + \textit{Additive Margin}$$
$$= 0.935 \times 1.05 + 0.02 = 0.98175 + 0.02 = 1.00175ns$$

The cell delay after AOCV adjustment, with guardband scaling and additive margin, is approximately 1.002 ns.

AOCV Numerical Examples

Problem 2: <u>Comparison With Traditional OCV</u>

Given:

- Nominal delay = 1ns
- Traditional OCV factor (fixed) = 1.20
- Using Advanced OCV:
  - Depth derate = 0.75 (at 8 stages)
  - Distance derate = 1.08 (at 400µm)
  - Guardband scale = 1.00
  - Additive margin = 0.01ns

Calculate:

- Traditional OCV derated delay.
- AOCV derated delay with guardband and margin.

AOCV Numerical Examples

Solution:

Traditional OCV derated delay:

$$1 \times 1.20 = 1.20ns$$

AOCV derated delay without guardband:

$$1 \times 0.75 \times 1.08 = 0.81ns$$

Including additive margin:

$$\text{Total Derate} = 0.81 \times 1.00 + 0.01 = 0.82ns$$

Interpretation:

AOCV reduces the pessimism compared to traditional OCV (0.82ns vs 1.20ns), improving timing margin accuracy.

AOCV Numerical Examples

Worst-case setup slack under AOCV is calculated as:

Setup slack = Clock period - (Clock-to-Q + Max. Logic Delay with AOCV + Setup time)

Worst-case hold slack under AOCV is:

Hold slack = (Clock-to-Q + Min. Logic Delay with AOCV) - Hold time

AOCV Numerical Examples

Problem-3: There are 2 registers separated by a combinational logic whose nominal logic max. delay is 4.8ns and min. delay is 1.1ns. The AOCV increases max. delay and min. delay by 15% and 25% respectively. The clock-to-Q delay of the register is 0.9ns and individually the setup and hold times are 0.6ns and 0.2ns. The clock frequency is 111.11MHz with no skew. Hereby, calculate the worst-case setup and hold slack considering AOCV and if there is any timing violation, suggest a design strategy as solution to the issue.

AOCV Numerical Examples

Given Data:

- Nominal max combinational delay = 4.8 ns
- Nominal min combinational delay = 1.1 ns
- AOCV increases max. delay by 15%
- AOCV increases min. delay by 25%
- Clock-to-Q delay = 0.9 ns
- Setup time = 0.6 ns
- Hold time = 0.2 ns
- Clock period = 9 ns
- Clock skew = 0 ns

Compute worst-case setup and hold slack considering AOCV

Step 1: Calculate AOCV adjusted delays

Max delay with AOCV:

$$\text{Max delay}_{AOCV} = 4.8 \times (1 + 0.15) = 4.8 \times 1.15 = 5.52\text{ns}$$

Min delay with AOCV:

$$\text{Min delay}_{AOCV} = 1.1 \times (1 + 0.25) = 1.1 \times 1.25 = 1.375\text{ns}$$

Step 2: Setup slack calculation (worst-case timing)

Setup slack is calculated as:

$$\text{Setup slack} = \text{Clock period} - [\text{Clock–to–Q delay} + \text{Max combinational delay}_{AOCV} + \text{Setup time}]$$

Substitute the values:

$$= 9 - [0.9 + 5.52 + 0.6] = 9 - 7.02 = 1.98\text{ns}$$

Step 3: Hold slack calculation (worst-case timing)

Hold slack is calculated as:

$$\text{Hold slack} = [\text{Clock–to–Q delay} + \text{Min combinational delay}_{AOCV}] - \text{Hold time} - \text{Clock skew}$$

Substitute the values:

$$= (0.9 + 1.375) - 0.2 - 0 = 2.075 - 0.2 = 1.875\text{ns}$$

AOCV Numerical Examples

Given Data:

- Nominal max combinational delay = 4.8 ns
- Nominal min combinational delay = 1.1 ns
- AOCV increases max. delay by 15%
- AOCV increases min. delay by 25%
- Clock-to-Q delay = 0.9 ns
- Setup time = 0.6 ns
- Hold time = 0.2 ns
- Clock period = 9 ns
- Clock skew = 0 ns

Violation Check

Setup slack = 1.98ns (positive slack) → No setup violation.
Hold slack = 1.875ns (positive slack) → No hold violation.
Thus, **no timing violations occur under AOCV assumptions**.

Recommended Design Strategy

Even though no violation occurs here, AOCV increases the combinational delays significantly, reducing timing margins compared to nominal. To ensure robust timing closure with AOCV:

- **Maintain timing slack buffers** during design to accommodate variations.
- **Optimize critical path logic** to reduce max delay, for example by gate sizing or restructuring.
- **Insert additional pipeline registers** if slack approaches zero to break long paths.
- **Use conservative estimation in clock period settings** if maximum throughput allows.
- **Employ AOCV-aware STA tools** to continuously monitor slack and ensure no margin erosion.

These design strategies ensure higher confidence in timing under process variation effects modeled by AOCV.

Advanced OCV (AOCV): Depth & Distance Aware Modeling

- AOCV introduces derate factors depending on “timing path depth” and “physical distance”, making timing analysis less pessimistic and more accurate compared to fixed OCV.
- Deep paths experience less overall variation; derate decreases as path depth increases.
- Depth computation and distance computation are tools-driven (e.g., PrimeTime AOCV setup commands).

PrimeTime AOCV Example:

```
set_aocvm_derate -analysis_type setup -arc_type cell -from FF1/CK -to FF2/D -derate_value 0.95 -depth 4
```

A 4-stage path, where base OCV derate is 0.95 for depth-4, actual adjusted delay = nominal_path_delay × derate.

Concept of Time Borrowing

- Time borrowing is a timing technique used mainly in latch-based digital circuits to allow combinational logic more time to compute by effectively "borrowing" slack time from the next clock phase or cycle.
- This happens because level-sensitive latches are transparent for a part of the clock period, so the data can arrive later than in edge-triggered flip-flop-based designs while still meeting timing constraints.
- Time borrowing permits a logic stage to use leftover time from the previous stage's clock period automatically, without additional circuit complexity or clock adjustments.
- It is primarily applicable in latch-based pipelines with two-phase clocking.
- This technique allows achieving higher throughput by accommodating longer combinational delays by passing slack from one stage to the next.
- Time borrowing extends permissible logic delay beyond a single clock cycle as long as the following stage's latch setup time constraints are met.

Setup and Hold Violation Fixing

Setup Violation Fixing

Setup violation occurs when the data arrives too late at the receiving register or latch, violating the setup time requirement before the clock edge.

Common fixing strategies include:

- Path delay reduction: Optimize logic by resizing gates, restructuring logic, or using faster cells.
- Pipeline refinement: Insert additional pipeline registers to reduce logic depth per stage.
- Clock frequency adjustment: Reduce clock frequency (increase clock period) to allow more time.
- Time borrowing: Use level-sensitive latches to borrow time from the next clock phase.
- Clock skew management: Introduce useful skew by adjusting clock arrival times to relax setup timing.

Setup and Hold Violation Fixing

Hold Violation Fixing

Hold violation occurs when data at the receiving register changes too soon after the clock edge, violating hold time.

Common fixing strategies include:

- Adding delay buffers: Insert delay elements (buffers, inverters) on the data path to increase delay.
- Reducing clock skew: Balance clock arrival times to minimize early data arrival.
- Logic restructuring: Modify or relocate logic gates to increase data path delay.
- Negative setup time strategies: If time borrowing is used, carefully assess hold requirements since time borrowing often does not help hold violations.

Setup and Hold Violation Fixing

Concept	Explanation	Fixing Strategies
Time Borrowing	Allows a latch-based pipeline stage to use leftover time from the previous clock cycle, improving timing margin.	Use level-sensitive latches with two-phase clocking.
Setup Violation Fixing	Data arrives late violating setup time.	Logic optimization, pipelining, clock adjustment, time borrowing, clock skew control.
Hold Violation Fixing	Data changes too early violating hold time.	Add delay buffers, clock skew tuning, logic restructuring.

Numerical Example: Problem-1

A synchronous circuit's critical path runs between FF1 and FF2, both on the rising clock edges. The circuit operates on clock frequency of 125MHz. The clock arrives at FF2, 0.5ns after FF1. The clock-to-Q travel time of FF1 is 1ns and the max. Datapath delay is given as 6ns. The setup time of FF2 is 1ns.

Calculate the setup slack and indicate if there is a timing violation. If the skew is intentionally increased to 1ns (i.e., delaying FF2 further), recalculate setup slack and discuss the effect of skew optimization.

Numerical Example: Problem-1 - Solution

The clock period for the synchronous circuit operating at 125 MHz is 8ns.

Original Setup Slack Calculation (Clock skew = 0.5ns)

- Clock-to-Q delay (FF1): 1ns
- Max. Datapath delay: 6ns
- Setup time (FF2): 1ns
- Clock skew (FF2 arrival delay): 0.5ns

Setup slack is calculated as:

$$\text{Setup Slack} = \textit{Clock Period} - \left(\textit{Clk-to-Q} + \textit{Datapath Delay} + \textit{Setup Time} + \textit{Skew}\right)$$
$$= 8 - (1 + 6 + 1 + 0.5) = 8 - 8.5 = -0.5ns$$

Since the setup slack is -0.5 ns, there is a timing violation (negative slack means the data does not arrive in time for the setup requirement).

Setup Slack with Increased Skew (Clock skew = 1.0 ns)

Recalculating with the increased skew:

$$\text{Setup Slack} = 8 - (1 + 6 + 1 + 1) = 8 - 9 = -1.0ns$$

The timing violation increases as the slack worsens to -1.0ns.

Effect of Skew Optimization

Increasing the positive skew (delay of FF2 clock arrival) worsens setup slack and thus makes the timing violation more severe. In this case, adding skew pushes the arrival of the clock at FF2 further, reducing the effective timing margin available for data to settle before the clock edge.

Skew optimization typically works to either reduce or redistribute skew to maximize timing slack. Here, increasing skew was detrimental, showing that for positive skew, careful balancing is needed to avoid setup violations.

Numerical Example: Problem-2

In a 2-phase latch-based pipeline, data passes through combinational logic (t_{delay}= 9ns) between Latch1 (transparent high) and Latch2 (transparent low). The clock frequency is 125MHz and Latch1 opens for 7ns each cycle. The hold and setup time for each latch is 0.4ns and the clock-to-Q delay is 1.2ns.

Calculate the maximum time data can "borrow" into the next clock phase without causing a setup or hold violation. If comb. logic delay rises to 11ns, show calculation of any resulting violation.

Numerical Example: Problem-2 - Solution

- Clock period at 125 MHz: 8.0ns
- Transparency (open time) of Latch1: 7.0ns
- Setup time per latch: 0.4ns
- Clock-to-Q delay: 1.2ns

Maximum Borrow Time

$$\text{Borrow time} = \textit{Latch open time} - (\textit{Clk-to-Q} + \textit{Setup time}) = 7.0 - (1.2 + 0.4) = 5.4\textit{ns}$$

This means that the data can extend, or "borrow," up to 5.4 ns into the next phase without violating the next latch's setup window.

Case 1: Logic Delay = 9ns

- Total window for data propagation: $8.0(\textit{clk period}) + 5.4(\textit{borrow}) = 13.4\textit{ns}$
- Slack after 9 ns logic delay: $13.4 - 9 = 4.4\textit{ns}$
- No setup violation: The slack is positive.
- No hold violation: $9 + 1.2 = 10.2$ ns, which is much greater than the 0.4 ns hold requirement.

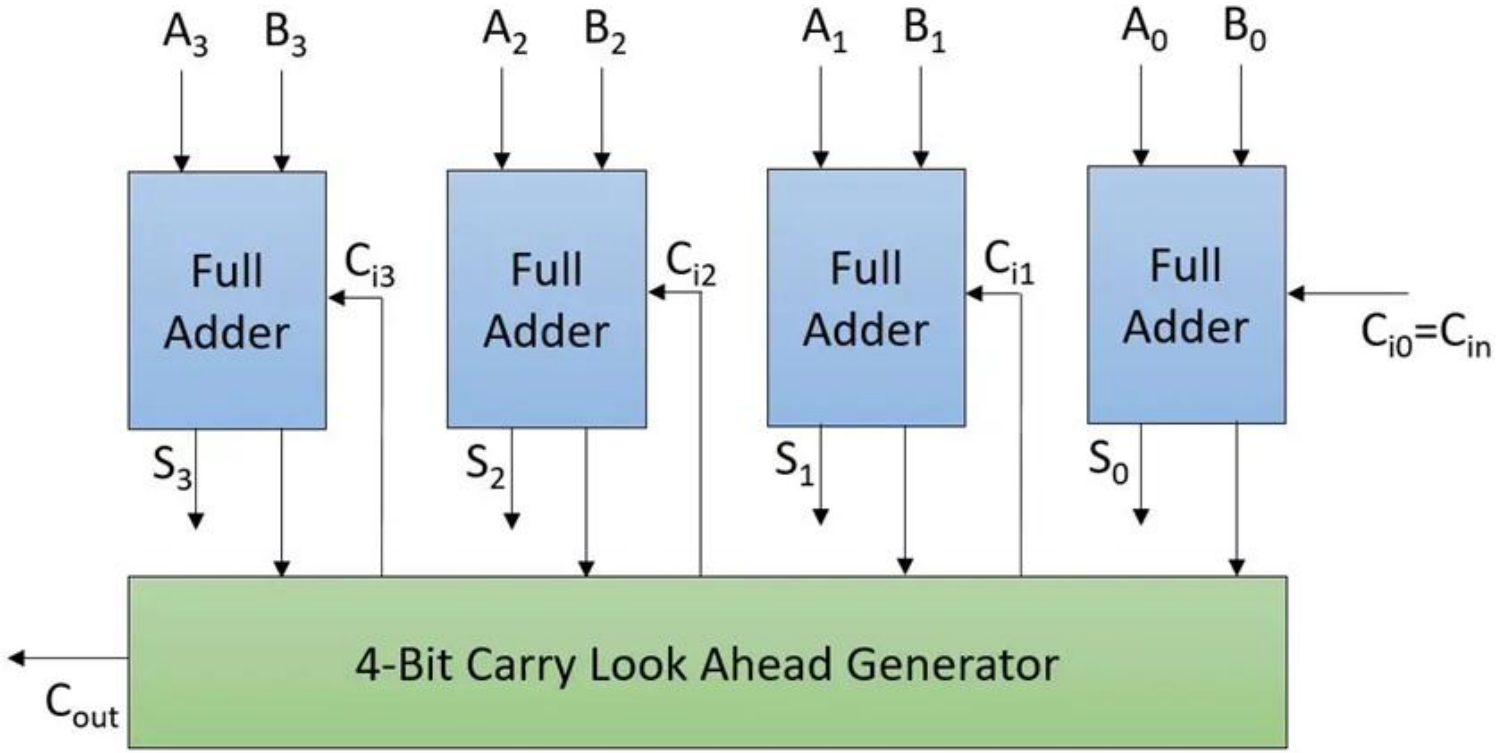
Case 2: Logic Delay = 11ns

- Total window for data propagation: $13.4\textit{ns}$
- Slack: $13.4 - 11 = 2.4\textit{ns}$
- No setup violation: The slack remains positive.
- No hold violation: $11 + 1.2 = 12.2$ns, still much greater than 0.4ns.

Miscellaneous Problem

A 4-bit carry look ahead adder is constructed using four full-adder (FA) cells, with each cell introducing a propagation delay of t_{FA}= 4ns. When two unsigned 4-bit inputs A=1001 and B=1011 is applied, and carry-in C_0=0 is provided at the least significant stage, calculate the total time for the output at the most significant sum bit (S_3) and its corresponding carry-out (C_4) to stabilize after the inputs transition. Write the transition timing description state delays through each stage.

Miscellaneous Problem

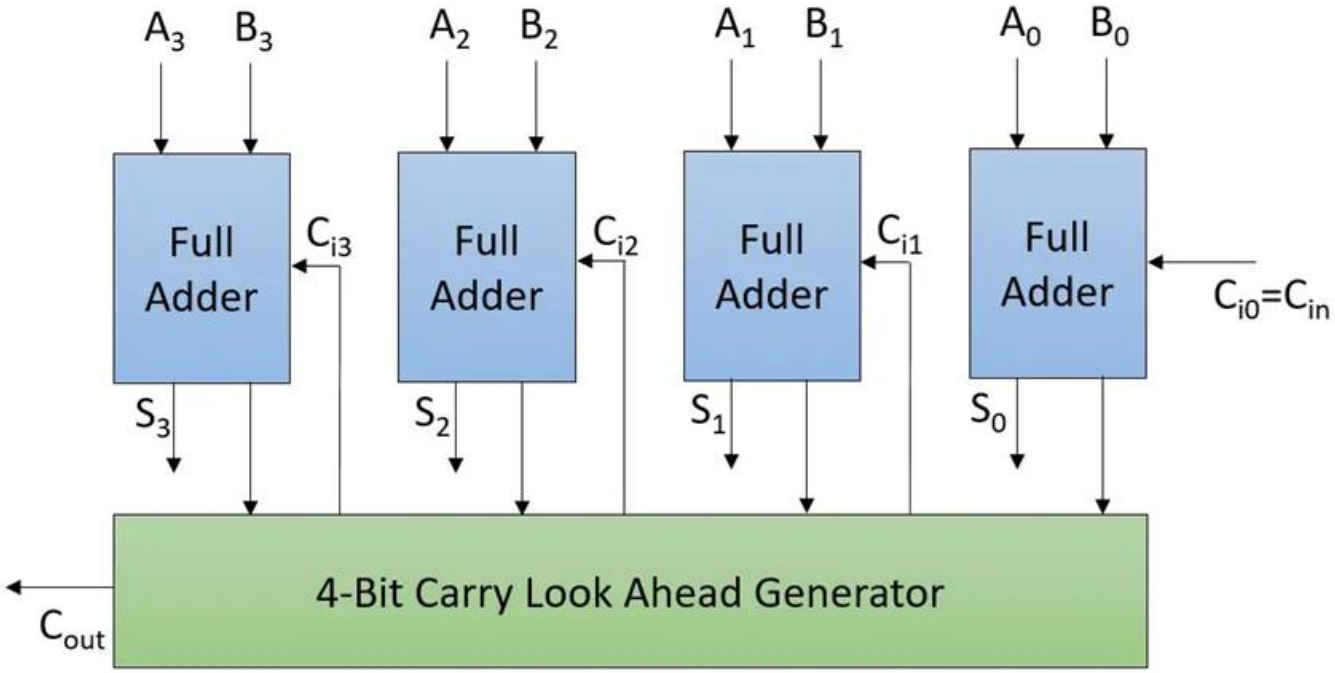


4-Bit Carry Look Ahead Adder

Carry Generate (G): This function denotes how the carry is generated for single-bit two inputs regardless of any input carry. The Carry Generate (G) is generated using the equation as A·B (similar to how carry is generated by full adder). Hence, **G = A·B**

Carry Propagate (P): This function denotes when the carry is propagated to the next stage with an addition whenever there is an input carry.

Miscellaneous Problem



4-Bit Carry Look Ahead Adder

Let's consider single bit two inputs A and B.

A	B	Carry In	Description
0	0	1	Carry is not propagated (0)
0	1	1	Carry is propagated (1)
1	0	1	Carry is propagated (1)
1	1	1	Carry is propagated (1)

Thus, P = A+B

Carry computation function for next stage

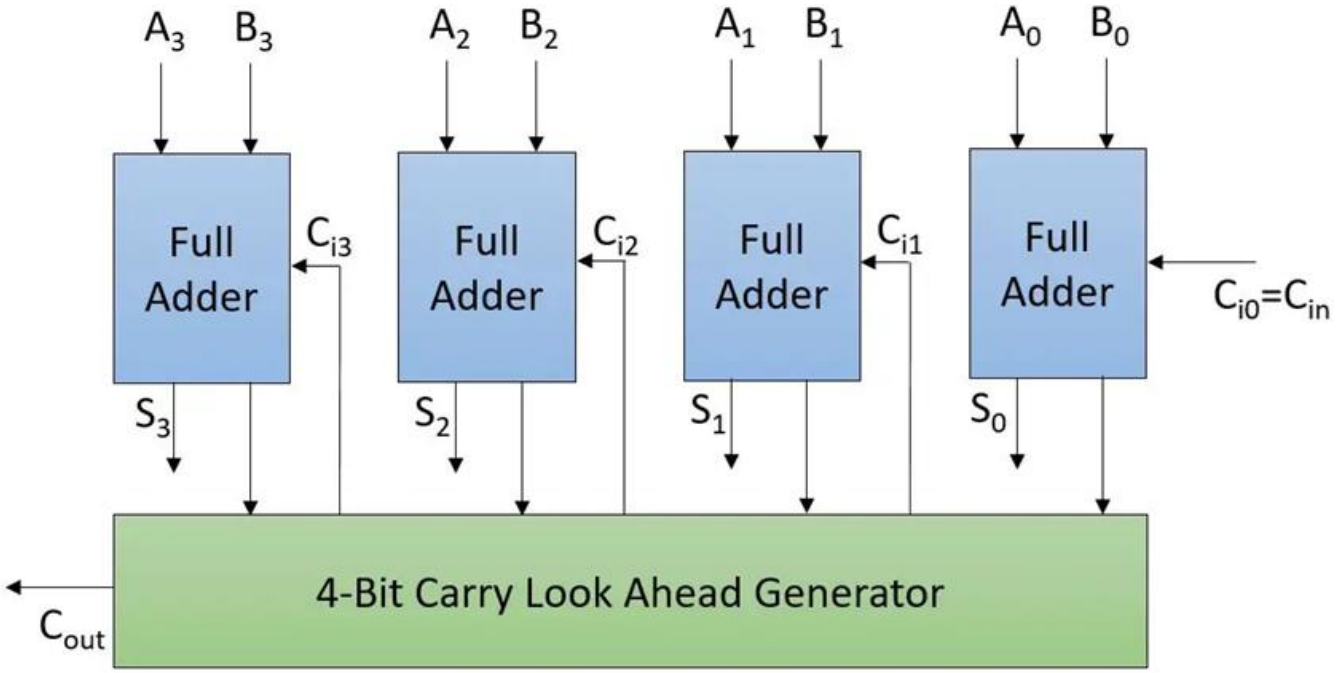
$C_{j+1} = G_j + (P_j \cdot C_j) = A_j \cdot B_j + (A_j + B_j) \cdot C_j$

However, if you notice clearly whenever Aj=1 and Bj=1

Cj+1 is (always) 1 as Aj·Bj component nullifies the effect of [(Aj + Bj)·Cj] part. Thus, it does not matter even if you use the equation propagate function as P = A (+) B.

G = A·B; P = A + B or A (+) B

Miscellaneous Problem



4-Bit Carry Look Ahead Adder

Inputs:

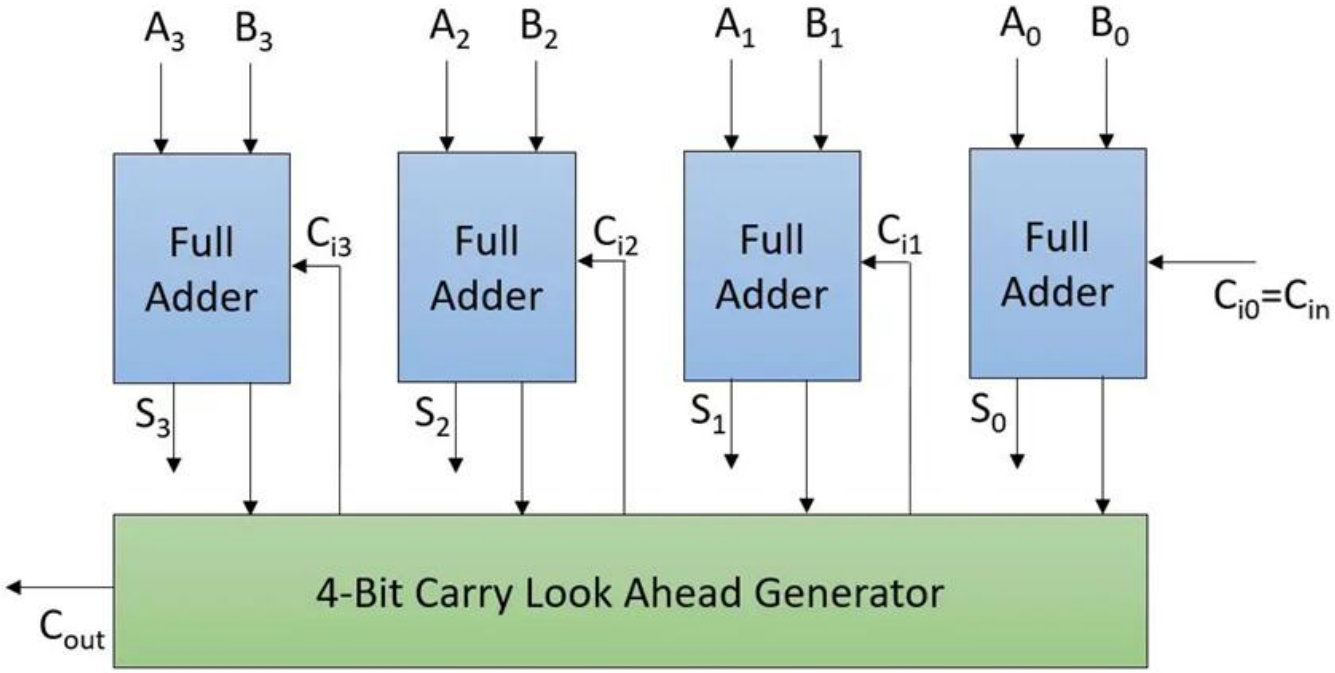
A3 A2 A1 A0

B3 B2 B1 B0 and Cin

Output:

Sum = S3 S2 S1 S0 and Cout

Miscellaneous Problem



4-Bit Carry Look Ahead Adder

Sum Sj = Pj ⊕ Cj = Aj ⊕ Bj ⊕ Cj

C0 = Cin

C1 = G0 + (P0·C0)

= A0·B0 + (A0⊕B0)·C0

C2 = G1 + (P1·C1) = A1·B1 + (A1⊕B1)·(A0·B0 + (A0⊕B0)·C0)

C3 = G2 + (P2·C2) = A2·B2 + (A2⊕B2)·C2 = A2·B2+(A2⊕B2)·(A1.B1+(A1⊕B1)·(A0B0 + (A0⊕B0)·C0))

C4 = G3 + (P3·C3) = A3·B3 + (A3⊕B3)·C3 = A3·B3 + (A3⊕B3)·(A2.B2 + (A2⊕B2)·(A1.B1 + (A1⊕B1)·(A0B0 + (A0⊕B0)·C0)))

Cout = C4;

Miscellaneous Problem – Solution

Timing Analysis for 4-bit Carry Look Ahead Adder (CLA)

- Each full adder (FA) has a propagation delay: $t_{FA} = 4$ ns
- Inputs: $A = 1001$, $B = 1011$, and carry-in $C_0 = 0$

Key Points:

- A CLA computes carry signals in parallel, reducing delay compared to ripple-carry adders.
- The delay for sum and carry outputs depends on the number of logic levels rather than the number of FA stages.

Total Delay Calculation

- CLA typically requires as many logic levels for carry generation as the number of bits in the adder, i.e., 4 levels for 4-bit.

- Each level corresponds to one FA cell delay, so total delay is:

$$\text{Total Delay} = 4 \times t_{FA} = 4 \times 4 \text{ ns} = 16 \text{ ns}$$

Transition Timing Description

Stage	Signal Stabilizes After (ns)	Justification
1st logic level	4	Initial propagate and generate signals from inputs stabilize
2nd logic level	8	Intermediate carries (e.g., C_1, C_2) stabilize
3rd logic level	12	Carry to later stages, sum bits like S_2 stabilize
4th logic level	16	Most significant sum bit S_3 and carry-out C_4 stabilize